MANAGED BY JIVAN JYOT TRUST, AMROLI.
PROF. V. B. SHAH INSTITUTE OF MANAGEMENT,
R. V. PATEL COLLEGE OF COMMERCE (ENG. MED.),
V. L. SHAH COLLEGE OF COMMERCE (GUJ. MED.) &
SUTEX BANK COLLEGE OF COMPUTER APPLICATIONS & SCIENCE
All Colleges are Affiliated to Veer Narmad South Gujarat University, Surat.
Self Financed B.B.A.-B.Com. (EM & GM)-B.C.A. (Regular/AI), B.Sc. (DS), M.Sc. (IT), M.Com. (EM & GM) Degree Programme
Accredited by National Assessment and Accreditation Council with "B" Grade
Dr. Mukesh R. Goyani
M.Com., M.Ed., M.Phil., GSET, Ph.D.
Principal
At & Po. : Amroli, Surat,
Pin. 394107, Gujarat (India)
www.amrolicollege.ac.in
(0261) 2494073, 2495643, 2496478
9427113947, 6354626492
principal@amrolicollege.ac.in
Principal_230@vnsgu.ac.in

PROJECT REPORT

ON

Analysis of VNSGU Student Academic Performance using Python

AS

A PARTIAL REQUIREMENT FOR THE DEGREE

OF

BACHELOR OF COMPUTER APPLICATION(BCA)

FOR SUBJECT

602 DATA ANALYSIS USING PYTHON

Submitted by:

DANKHARA DHRUV NARESHBHAI (6093)

LIMBANI OM JAGDISHBHAI (6242)

Guided by:

Asst. Prof. Twinkle S. Panchal

Sutex Bank College of Computer Applications and Science, Amroli

MANAGED BY JIVAN JYOT TRUST, AMROLI.
PROF. V. B. SHAH INSTITUTE OF MANAGEMENT,
R. V. PATEL COLLEGE OF COMMERCE (ENG. MED.),
V. L. SHAH COLLEGE OF COMMERCE (GUJ. MED.) &
SUTEX BANK COLLEGE OF COMPUTER APPLICATIONS & SCIENCE
All Colleges are Affiliated to Veer Narmad South Gujarat University, Surat.
Self Financed B.B.A.-B.Com. (EM & GM)-B.C.A. (Regular/AI), B.Sc. (DS), M.Sc. (IT), M.Com. (EM & GM) Degree Programme
Accredited by National Assessment and Accreditation Council with "B" Grade

Dr. Mukesh R. Goyani
M.Com., M.Ed., M.Phil., GSET, Ph.D.
Principal

At & Po. : Amroli, Surat,
Pin. 394107, Gujarat (India)
www.amrolicollege.ac.in

(0261) 2494073, 2495643, 2496478
9427113947, 6354626492
principal@amrolicollege.ac.in
Principal_230@vnsgu.ac.in

Certificate

Bachelor of Computer Applications

This is to certify that **Mr./Ms. DANKHARA DHRUV NARESHBHAI** (Exam No: **6093** & Roll No: **616**) has successfully completed his/her **Minor Project** Work entitled **BCA SEM 4 RESULT ANALYSIS** for the Subject: **602 - Data Analytics using Python** of the 6th Semester during the academic year 2025-26 under **Sutex Bank College of Computer Applications & Science** affiliated to **Veer Narmad South Gujarat University, Surat**.

Group No. : TY6-24

Internal Guide : Ms. Twinkle S. Panchal

Signature :

Date : 30.03.2026

External Examiner :

Dr. Mukesh R. Goyani
Principal

MANAGED BY JIVAN JYOT TRUST, AMROLI.
PROF. V. B. SHAH INSTITUTE OF MANAGEMENT,
R. V. PATEL COLLEGE OF COMMERCE (ENG. MED.),
V. L. SHAH COLLEGE OF COMMERCE (GUJ. MED.) &
SUTEX BANK COLLEGE OF COMPUTER APPLICATIONS & SCIENCE
All Colleges are Affiliated to Veer Narmad South Gujarat University, Surat.
Self Financed B.B.A.-B.Com. (EM & GM)-B.C.A. (Regular/AI), B.Sc. (DS), M.Sc. (IT), M.Com. (EM & GM) Degree Programme
Accredited by National Assessment and Accreditation Council with "B" Grade

Dr. Mukesh R. Goyani
M.Com., M.Ed., M.Phil., GSET, Ph.D.
Principal

At & Po. : Amroli, Surat,
Pin. 394107, Gujarat (India)
www.amrolicollege.ac.in

(0261) 2494073, 2495643, 2496478
9427113947, 6354626492
principal@amrolicollege.ac.in
Principal_230@vnsgu.ac.in

Certificate

Bachelor of Computer Applications

This is to certify that **Mr./Ms. LIMBANI OM JAGDISHBHAI** (Exam No: **6242** & Roll No: **635**) has successfully completed his/her **Minor Project** Work entitled **BCA SEM 4 RESULT ANALYSIS** for the Subject: **602 - Data Analytics using Python** of the 6th Semester during the academic year 2025-26 under **Sutex Bank College of Computer Applications & Science** affiliated to **Veer Narmad South Gujarat University, Surat**.

Group No. : TY6-24

Internal Guide : Ms. Twinkle S. Panchal

Signature :

Date : 30.03.2026

Dr. Mukesh R. Goyani
Principal

External Examiner :

To be used for academic certification only.

Acknowledgement

We would like to express our sincere gratitude to **Asst. Prof. Twinkle S. Panchal**, Sutex Bank College of Computer Applications and Science, Amroli, for her valuable guidance, continuous encouragement, and constant support throughout the development of this project titled **“Analysis of VNSGU Student Academic Performance using Python”**, submitted as a part of the subject **Data Analysis Using Python (602)**.

Her expert knowledge, constructive suggestions, and thoughtful feedback helped us understand the concepts of data analysis, exploratory data analysis, visualization techniques, and machine learning implementation more effectively. Her motivation and guidance played an important role in successfully completing this project and improving our understanding of practical data analytics using Python tools and libraries.

We are also thankful to **Sutex Bank College of Computer Applications and Science, Amroli**, for providing us with the necessary academic environment, infrastructure, and learning resources required to carry out this project work successfully. The support from the college helped us gain practical exposure to real-world data analysis techniques and strengthened our knowledge in the field of data science and analytics.

This project provided us with an opportunity to apply theoretical concepts such as data preprocessing, statistical analysis, data visualization, and machine learning clustering techniques on real academic datasets. Through this work, we were able to develop a deeper understanding of how data-driven approaches can help in identifying patterns, trends, and insights in educational data.

Finally, we would like to extend our sincere thanks to everyone who directly or indirectly supported and encouraged us during the completion of this project. Their support and motivation contributed significantly to the successful completion of this work.

DANKHARA DHRUV NARESHBHAI

LIMBANI OM JAGDISHBHAI

INDEX

Sr. No.	Chapter Title	Page No
1.	Project Definition	01
2.	Dataset Details	02
3.	Python tools and libraries used	03
4.	Understanding data	04
5.	Understanding Spread of Data	08
6.	Automating EDA using Python	10
7.	Handling missing data and outliers	13
8.	Univariate Analysis	16
9.	Bivariate Analysis	20
10.	Multivariate Analysis	25
11.	Data Visualization	31
12.	Machine Learning Implementation	38
13.	Conclusion	47
14.	Reference	48

1.Project Definition

Analysis of VNSGU Student Academic Performance using Python

Understanding student academic performance is essential for evaluating educational quality, identifying learning gaps, and improving institutional decision-making. In this project, we analyse the VNSGU student result dataset using Python to extract meaningful academic insights through Exploratory Data Analysis (EDA).

This project focuses on analysing structured examination result data to understand performance distribution, subject-wise trends, grading patterns, and overall academic outcomes.

The primary objective of this analysis is to transform raw examination result data into actionable insights using statistical techniques and data visualization tools.

Project Objectives :

We aim to answer the following analytical questions:

1. What is the overall performance distribution of students?
2. What are the average marks obtained in each subject?
3. Which subject has the highest failure rate?
4. How are total marks distributed among students?
5. What percentage of students passed and failed?
6. Identify top-performing students based on total marks.
7. Are there any outliers in subject-wise marks?
8. What is the correlation between subjects and total marks?

2.Dataset Details

Dataset Name: VNSGU Master Student Result Dataset

Dataset Source:

https://ums.vnsgu.net/Result/StudentResultDisplay.aspx?HtmlURL=6784,0000

Dataset Type: Structured CSV File(.csv)

Dataset Domain: Educational Data Analytics / Academic Performance Analysis

Dataset Description

The dataset contains academic performance records of students enrolled under VNSGU. It includes structured information related to student identification, subject-wise marks, total marks, and result status.

The dataset typically contains the following types of attributes:

- Seat Number (Unique Student Identifier)
- Student Name
- Collage Name
- Subject-wise Marks
- Total Marks
- SGPA
- Percentage
- Result Status (Pass/Fail)

3.Python Tools and Libraries Used

Python 3.10

Python is a widely used programming language in data science due to its simplicity and powerful libraries. It supports mathematical operations, data analysis, and visualization efficiently. Its easy syntax and flexibility make it suitable for handling large datasets and performing complex computations.

Pandas

Pandas is an open-source library used for data manipulation and analysis. It provides data structures like DataFrames and Series to handle tabular data efficiently. It supports operations such as data cleaning, filtering, grouping, and file handling (CSV, Excel), making it essential for data analysis.

NumPy

NumPy is a core library for numerical computations in Python. It provides fast multi-dimensional arrays and mathematical functions. It is widely used for performing calculations and serves as the foundation for libraries like Pandas and Scikit-learn.

Matplotlib

Matplotlib is a powerful library used for data visualization. It helps create graphs such as bar charts, line plots, scatter plots, and histograms. It provides customization options to present data clearly and effectively.

Seaborn

Seaborn is a high-level visualization library built on Matplotlib. It provides attractive and informative statistical plots like heatmaps, violin plots, and distribution graphs. It simplifies data visualization and is useful for exploratory data analysis.

Scikit-learn

Scikit-learn is a machine learning library used for predictive analysis. It provides tools for classification, regression, clustering, and model evaluation. It is widely used in data science for building and analysing models.

4.Understanding data

4.1: Importing necessary Python libraries :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

4.2: Creating the data frame :

```
# Load dataset from CSV file
file_path = "D:\\VNSGU_PDA2\\Data-Analytics-using-\
Python-Minor-Project\\Data\\raw\\VNSGU_SEM4_\
All_Student_Results.csv"

df = pd.read_csv(file_path)
```

4.3 display all column names :

```
print(df.columns)
```

✓ 0.0s

```
Index(['Seat Number', 'Name', 'College', 'Total Marks', 'SGPA', 'Result',
       'JAVA_PROGRAMMING_LANGUAGE_Ext', 'JAVA_PROGRAMMING_LANGUAGE_Int',
       'JAVA_PROGRAMMING_LANGUAGE_Total',
       'JAVA_PROGRAMMING_LANGUAGE_Theory_Ext',
       'JAVA_PROGRAMMING_LANGUAGE_Theory_Int',
       'JAVA_PROGRAMMING_LANGUAGE_Practical_Ext',
       'JAVA_PROGRAMMING_LANGUAGE_Practical_Int', '.NET_PROGRAMMING_Ext',
       '.NET_PROGRAMMING_Int', '.NET_PROGRAMMING_Total',
       '.NET_PROGRAMMING_Theory_Ext', '.NET_PROGRAMMING_Theory_Int',
       '.NET_PROGRAMMING_Practical_Ext', '.NET_PROGRAMMING_Practical_Int',
       'MOBILE_APPLICATION_DEVELOPMENT_-_II_Ext',
       'MOBILE_APPLICATION_DEVELOPMENT_-_II_Int',
       'MOBILE_APPLICATION_DEVELOPMENT_-_II_Total',
       'MOBILE_APPLICATION_DEVELOPMENT_-_II_Theory_Ext',
       'MOBILE_APPLICATION_DEVELOPMENT_-_II_Theory_Int',
       'MOBILE_APPLICATION_DEVELOPMENT_-_II_Practical_Ext',
       'MOBILE_APPLICATION_DEVELOPMENT_-_II_Practical_Int',
       'INTERNET_OF_THINGS_(IOT)_Ext', 'INTERNET_OF_THINGS_(IOT)_Int',
       'INTERNET_OF_THINGS_(IOT)_Total',
       'ORGANIZATIONAL_SOFT-SKILLS_IN_SOFTWARE_INDUSTRY_Ext',
       'ORGANIZATIONAL_SOFT-SKILLS_IN_SOFTWARE_INDUSTRY_Int',
       'ORGANIZATIONAL_SOFT-SKILLS_IN_SOFTWARE_INDUSTRY_Total',
       'CERTIFICATE_COURSE_IN_WEB_DESIGNING_(HTML,JAVA_SCRIPT,CSS)_Ext',
       'CERTIFICATE_COURSE_IN_WEB_DESIGNING_(HTML,JAVA_SCRIPT,CSS)_Int',
       'CERTIFICATE_COURSE_IN_WEB_DESIGNING_(HTML,JAVA_SCRIPT,CSS)_Total',
```

```
  'BHARATIYA_MULYA_PARAMPARA_-_II_Ext',
  'BHARATIYA_MULYA_PARAMPARA_-_II_Int',
  'BHARATIYA_MULYA_PARAMPARA_-_II_Total', 'Course_Name'],
dtype='object')
```

4.5 display top few lines of data :

```
df.head(5)
```

```
    Seat Number                       Name  \
0          1007  AHIR VIRALKUMAR RAMESHBHAI
1          1005       AHIR NILAY ARVINDBHAI
2          1004    AHIR MITKUMAR SURESHBHAI
3          1002      AHIR KRISH PRAMODKUMAR
4          1006       AHIR TEJ PRAKASHKUMAR

                                       College Total Marks  SGPA  \
0  V.S. PATEL COLLEGE OF ARTS & SCIENCE, BILIMORA   266 / 550  4.82
1  V.S. PATEL COLLEGE OF ARTS & SCIENCE, BILIMORA   306 / 550  5.73
2  V.S. PATEL COLLEGE OF ARTS & SCIENCE, BILIMORA   333 / 550  5.91
3  V.S. PATEL COLLEGE OF ARTS & SCIENCE, BILIMORA   297 / 550  5.27
4  V.S. PATEL COLLEGE OF ARTS & SCIENCE, BILIMORA   325 / 550  5.82

       Result JAVA_PROGRAMMING_LANGUAGE_Ext JAVA_PROGRAMMING_LANGUAGE_Int  \
0  PASS O.150                            19                            22
1        PASS                            21                            24
2        PASS                            26                            24
3        PASS                            19                            20
4        PASS                            23                            24

  JAVA_PROGRAMMING_LANGUAGE_Total JAVA_PROGRAMMING_LANGUAGE_Theory_Ext  ...  \
0                              41                                   09  ...
1                              45                                   11  ...
2                              50                                   16  ...
...
3  CERTIFICATE COURSE IN WEB DESIGNING (HTML,JAVA...
4  CERTIFICATE COURSE IN WEB DESIGNING (HTML,JAVA...
```

4.6 display shape of data :

```
df.shape
```

```
(9157, 40)
```

4.7 Getting summary of the dataframe using df.info() :

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9157 entries, 0 to 9156
Data columns (total 40 columns):
 #   Column                                Non-Null Count  Dtype
---  ------                                --------------  -----
 0   Seat Number                           9157 non-null   int64
 1   Name                                  9157 non-null   object
 2   College                               9157 non-null   object
 3   Total Marks                           9157 non-null   object
 4   SGPA                                  9157 non-null   object
 5   Result                                9157 non-null   object
 6   JAVA_PROGRAMMING_LANGUAGE_Ext         9157 non-null   object
 7   JAVA_PROGRAMMING_LANGUAGE_Int         9157 non-null   object
 8   JAVA_PROGRAMMING_LANGUAGE_Total       9157 non-null   object
 9   JAVA_PROGRAMMING_LANGUAGE_Theory_Ext  9157 non-null   object
 10  JAVA_PROGRAMMING_LANGUAGE_Theory_Int  9157 non-null   object
 11  JAVA_PROGRAMMING_LANGUAGE_Practical_Ext  9157 non-null   object
 12  JAVA_PROGRAMMING_LANGUAGE_Practical_Int  9157 non-null   object
 13  .NET_PROGRAMMING_Ext                  9157 non-null   object
 14  .NET_PROGRAMMING_Int                  9157 non-null   int64
 15  .NET_PROGRAMMING_Total                9157 non-null   object
 16  .NET_PROGRAMMING_Theory_Ext           9157 non-null   object
 17  .NET_PROGRAMMING_Theory_Int           9157 non-null   int64
 18  .NET_PROGRAMMING_Practical_Ext        9157 non-null   object
 19  .NET_PROGRAMMING_Practical_Int        9157 non-null   object
...
 39  Course_Name                           7554 non-null   object
dtypes: int64(6), object(34)
```

5. Understanding Spread of data

With our dataset we have used histogram with “kde=True” to understand the distribution and spread of data in column “rate” (user ratings).

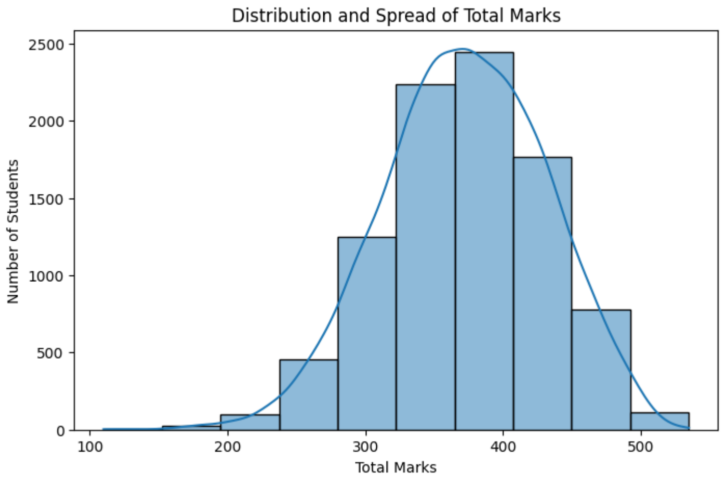
5.1 Distribution of Total Marks (Histogram)

To understand how students performed overall, we visualize the distribution of total marks using a histogram with kernel density estimation (KDE).

A histogram shows:

- How marks are distributed
- Which score range has maximum students
- Whether distribution is normal, skewed, or uneven

```
# Plot distribution of Total Marks
plt.figure(figsize=(8, 5))

sns.histplot(
    df['Total Marks'],
    bins=10,
    kde=True
)

plt.title("Distribution and Spread of Total Marks")
plt.xlabel("Total Marks")
plt.ylabel("Number of Students")

plt.show()
```



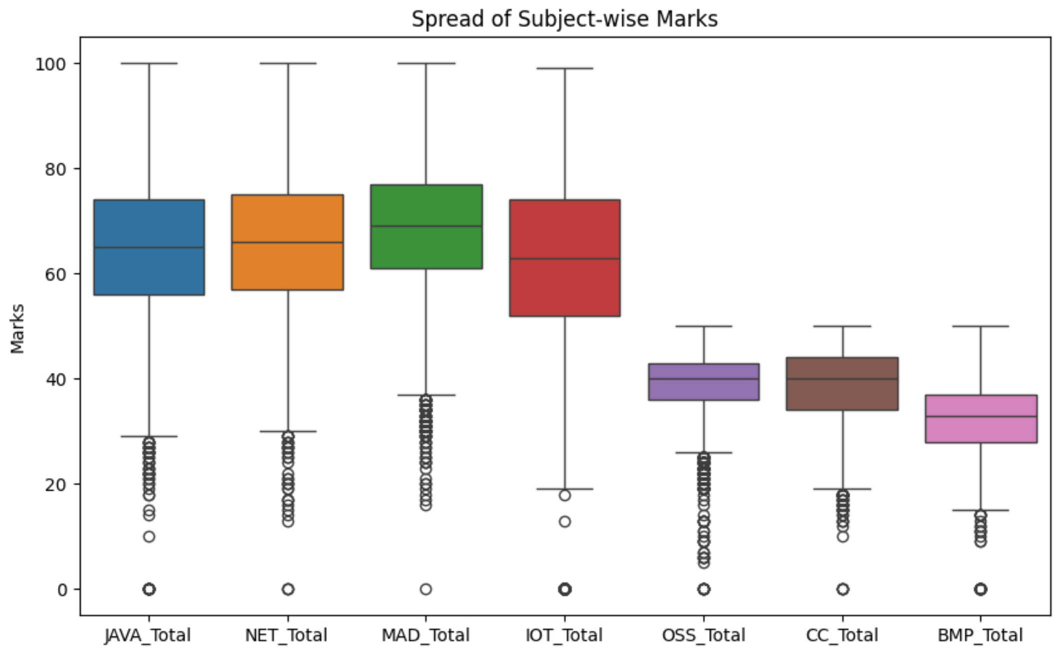
5.2 Spread of Subject-wise Marks (Boxplot Analysis)

To understand variability and detect outliers in individual subjects, we use boxplots.

A boxplot helps in:

- Identifying median marks
- Observing spread (IQR)
- Detecting outliers
- Comparing subject difficulty levels

```
# Boxplot for subject-wise marks distribution
plt.figure(figsize=(10, 6))

sns.boxplot(
    data=df[["JAVA_Total", "NET_Total", "MAD_Total",
             "IOT_Total", "OSS_Total", "CC_Total",
             "BMP_Total"]]
)

plt.title("Spread of Subject-wise Marks")
plt.ylabel("Marks")

plt.show()
```



6. Automating EDA using python

Automating Exploratory Data Analysis (EDA) in Python is achieved using specialized libraries that generate comprehensive, interactive reports with just a few lines of code.

In Python's pandas library, df.info() and df.describe() are fundamental methods for performing initial exploratory data analysis (EDA).

df.info(): Provides a concise structural summary of the DataFrame. It provides following information about dataset:

- Data Types (Dtypes),
- Non-Null Counts,
- Column Names,
- Memory Usage,
- Total Entries/Rows

df.describe(): Offers a statistical summary of the numerical data.

- Count: The number of non-null observations in each column.
- Mean: The average value.
- Std: The standard deviation (spread of data).
- Min: The minimum value.
- 25%, 50%, 75%: The first quartile, median (50th percentile), and third quartile.
- Max: The maximum value.

These functions help data analysts quickly understand the dataset's composition and spot potential issues like missing values or outliers without manually sifting through rows.

6.1: Getting summary of the dataframe using df.info()

```
print("---------- Dataset Structural Summary ----------")
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9157 entries, 0 to 9156
Data columns (total 40 columns):
 #   Column                                 Non-Null Count  Dtype
---  ------                                 --------------  -----
 0   Seat Number                            9157 non-null   int64
 1   Name                                   9157 non-null   object
 2   College                                9157 non-null   object
 3   Total Marks                            9157 non-null   object
 4   SGPA                                   9157 non-null   object
 5   Result                                 9157 non-null   object
 6   JAVA_PROGRAMMING_LANGUAGE_Ext          9157 non-null   object
 7   JAVA_PROGRAMMING_LANGUAGE_Int          9157 non-null   object
 8   JAVA_PROGRAMMING_LANGUAGE_Total        9157 non-null   object
 9   JAVA_PROGRAMMING_LANGUAGE_Theory_Ext   9157 non-null   object
 10  JAVA_PROGRAMMING_LANGUAGE_Theory_Int   9157 non-null   object
 11  JAVA_PROGRAMMING_LANGUAGE_Practical_Ext 9157 non-null  object
 12  JAVA_PROGRAMMING_LANGUAGE_Practical_Int 9157 non-null  object
 13  .NET_PROGRAMMING_Ext                   9157 non-null   object
 14  .NET_PROGRAMMING_Int                   9157 non-null   int64
 15  .NET_PROGRAMMING_Total                 9157 non-null   object
 16  .NET_PROGRAMMING_Theory_Ext            9157 non-null   object
 17  .NET_PROGRAMMING_Theory_Int            9157 non-null   int64
 18  .NET_PROGRAMMING_Practical_Ext         9157 non-null   object
 19  .NET_PROGRAMMING_Practical_Int         9157 non-null   object
...
 39  Course_Name                            7554 non-null   object
dtypes: int64(6), object(34)
```

6.2: Getting description of the dataframe using df.describe()

```
print("---------- Statistical Summary ----------")
print(df.describe())
```

```
        Seat Number  Total Marks   Percentage         SGPA     JAVA_Ext  \
count   9157.000000  9157.000000  9157.000000  9157.000000  9157.000000
mean    5618.745768   372.151906    67.663983     6.400311    29.912526
std     2669.705133    59.295789    10.781053     1.949091     8.609654
min     1001.000000   110.000000    20.000000     0.000000     0.000000
25%     3302.000000   332.000000    60.363636     6.000000    25.000000
50%     5615.000000   373.000000    67.818182     6.730000    30.000000
75%     7929.000000   415.000000    75.454545     7.550000    35.000000
max    10249.000000   535.000000    97.272727    10.000000    50.000000

          JAVA_Int   JAVA_Total  JAVA_Th_Ext  JAVA_Th_Int  JAVA_Pr_Ext  ...  \
count  9157.000000  9157.000000  9157.000000  9157.000000  9157.000000  ...
mean     34.600415    64.881293    13.239271    17.068800    17.059954  ...
std       6.962492    13.165197     4.527750     3.811651     4.781783  ...
min       0.000000     0.000000     0.000000     0.000000     0.000000  ...
25%      30.000000    56.000000    10.000000    15.000000    14.000000  ...
50%      35.000000    65.000000    13.000000    17.000000    18.000000  ...
75%      40.000000    74.000000    16.000000    20.000000    20.000000  ...
max      50.000000   100.000000    25.000000    25.000000    25.000000  ...

         IOT_Total      OSS_Ext      OSS_Int    OSS_Total      WEB_Ext  \
count  9157.000000  9157.000000  9157.000000  9157.000000  9157.000000
mean     62.132139    19.586983    19.325216    38.912417    19.299771
std      16.152256     3.430232     3.293219     5.769375     4.458538
min       0.000000     0.000000     0.000000     0.000000     0.000000
...
75%      22.000000    44.000000    17.000000    21.000000    37.000000
max      25.000000    50.000000    25.000000    25.000000    50.000000
```

7. Handling missing data and outliers data

Before performing deeper analysis, it is essential to clean and prepare the dataset. Raw academic data may contain:

- Missing values
- Incorrect data types
- Duplicate records
- Extreme values (outliers)

Data cleaning ensures accurate and reliable analysis results.

7.1 Handling Missing Values :

Missing values can distort statistical calculations and visualizations. Therefore, we first identify and handle them properly.

```
print("---------- Checking Missing Values ----------")
print(df.isnull().sum())
```

```
Name                                          0
College                                       0
Total Marks                                   0
SGPA                                          0
Result                                        0
JAVA_PROGRAMMING_LANGUAGE_Ext                 0
JAVA_PROGRAMMING_LANGUAGE_Int                 0
JAVA_PROGRAMMING_LANGUAGE_Total               0
JAVA_PROGRAMMING_LANGUAGE_Theory_Ext          0
JAVA_PROGRAMMING_LANGUAGE_Theory_Int          0
JAVA_PROGRAMMING_LANGUAGE_Practical_Ext       0
JAVA_PROGRAMMING_LANGUAGE_Practical_Int       0
.NET_PROGRAMMING_Ext                          0
.NET_PROGRAMMING_Int                          0
.NET_PROGRAMMING_Total                        0
.NET_PROGRAMMING_Theory_Ext                   0
.NET_PROGRAMMING_Theory_Int                   0
.NET_PROGRAMMING_Practical_Ext                0
.NET_PROGRAMMING_Practical_Int                0
MOBILE_APPLICATION_DEVELOPMENT_-_II_Ext       0
MOBILE_APPLICATION_DEVELOPMENT_-_II_Int       0
MOBILE_APPLICATION_DEVELOPMENT_-_II_Total     0
MOBILE_APPLICATION_DEVELOPMENT_-_II_Theory_Ext 0
MOBILE_APPLICATION_DEVELOPMENT_-_II_Theory_Int 0
...
BHARATIYA_MULYA_PARAMPARA_-_II_Ext            0
BHARATIYA_MULYA_PARAMPARA_-_II_Int            0
BHARATIYA_MULYA_PARAMPARA_-_II_Total          0
```

7.2: Fill Missing Values :

7.2.1 Handling SGPA Column :

The SGPA column contained the value "--" for students who failed the examination. Since this does not represent zero SGPA but rather an undefined grade point, these values were converted to NaN instead of zero to maintain statistical integrity.

```
df['SGPA'] = df['SGPA'].replace('--', np.nan)
df['SGPA'] = pd.to_numeric(df['SGPA'], errors='coerce')
```

7.2.2 Cleaning Total Marks Column :

The “Total Marks” column contained values in the format “Obtained Marks / Maximum Marks” (e.g., 345/550). Since only the obtained marks were required for analysis, the denominator part was removed.

```
df['Total Marks'] = df['Total Marks'].astype(str).str.split('/').str[0].str.strip()
df['Total Marks'] = pd.to_numeric(df['Total Marks'], errors='coerce')
```

7.2.3 Converting Marks Columns to Numeric :

The subject mark columns contained some non-numeric values and blank entries. Since these represented either absent students or zero marks, they were converted into numeric format and missing values were replaced with 0.

```
df[marks_columns] = (
    df[marks_columns]
    .apply(pd.to_numeric, errors='coerce')
    .fillna(0)
    .astype(int)
)
```

8. Univariate Analysis

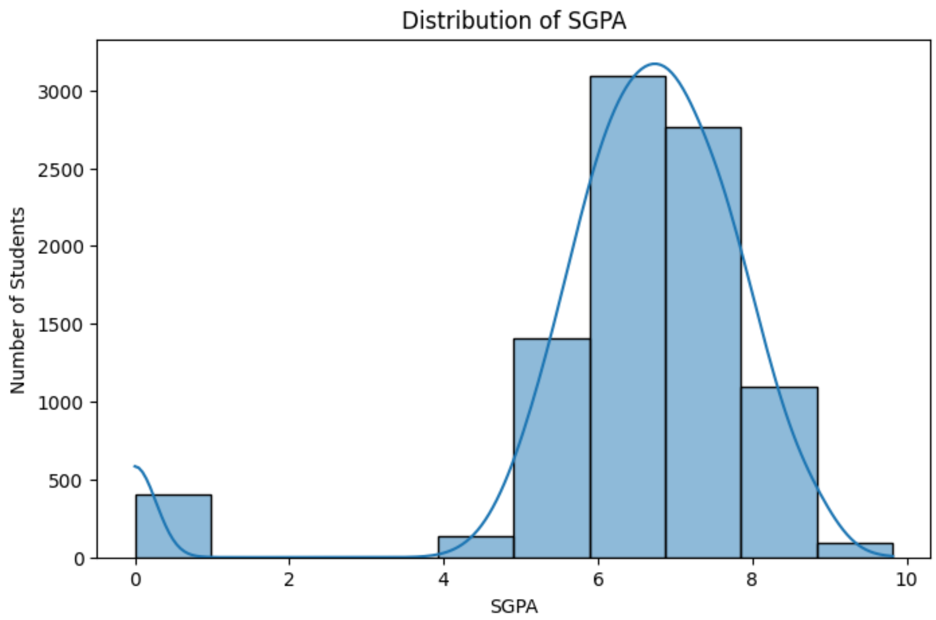
Univariate Analysis involves analysing a single variable at a time to understand its distribution, central tendency, and frequency patterns.

In this chapter, we analyse:

- SGPA Distribution
- Pass vs Fail Ratio
- Subject-wise Failure Count
- Percentage Distribution

8.1 SGPA Distribution :

```
plt.figure(figsize=(8,5))
sns.histplot(df['SGPA'], bins=10, kde=True)
plt.title("Distribution of SGPA")
plt.xlabel("SGPA")
plt.ylabel("Number of Students")
plt.show()
```

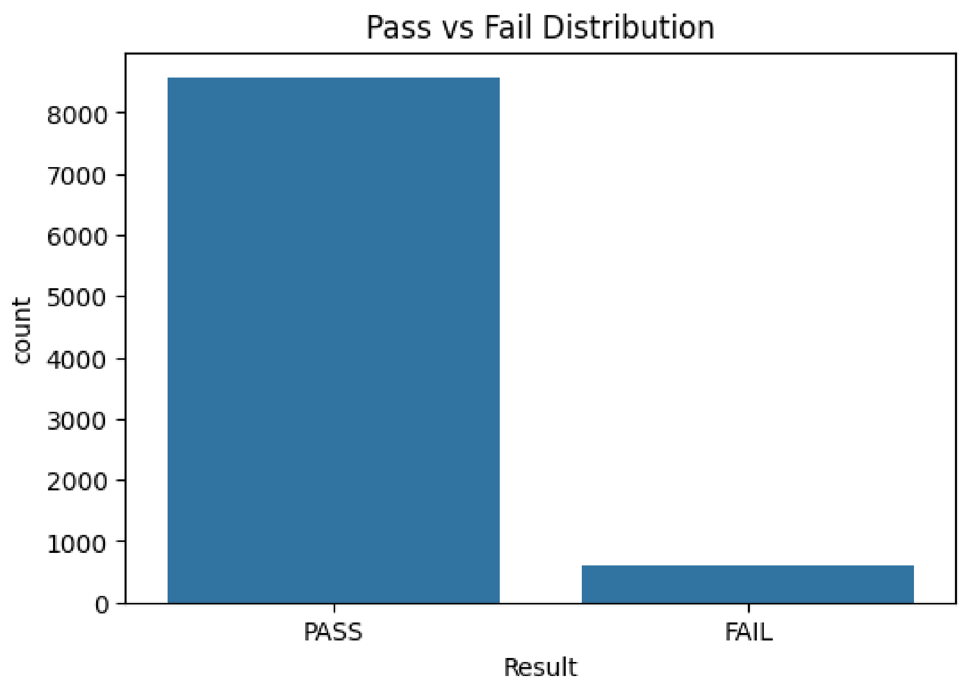


Interpretation:

- Majority clustered around 6–8 average performance.
- High peak near 8–9 strong academic performance.
- Values near 0 indicate failed students (if included).

8.2 Pass vs Fail Distribution :

```
plt.figure(figsize=(6,4))
sns.countplot(x='Result', data=df)
plt.title("Pass vs Fail Distribution")
plt.show()
```

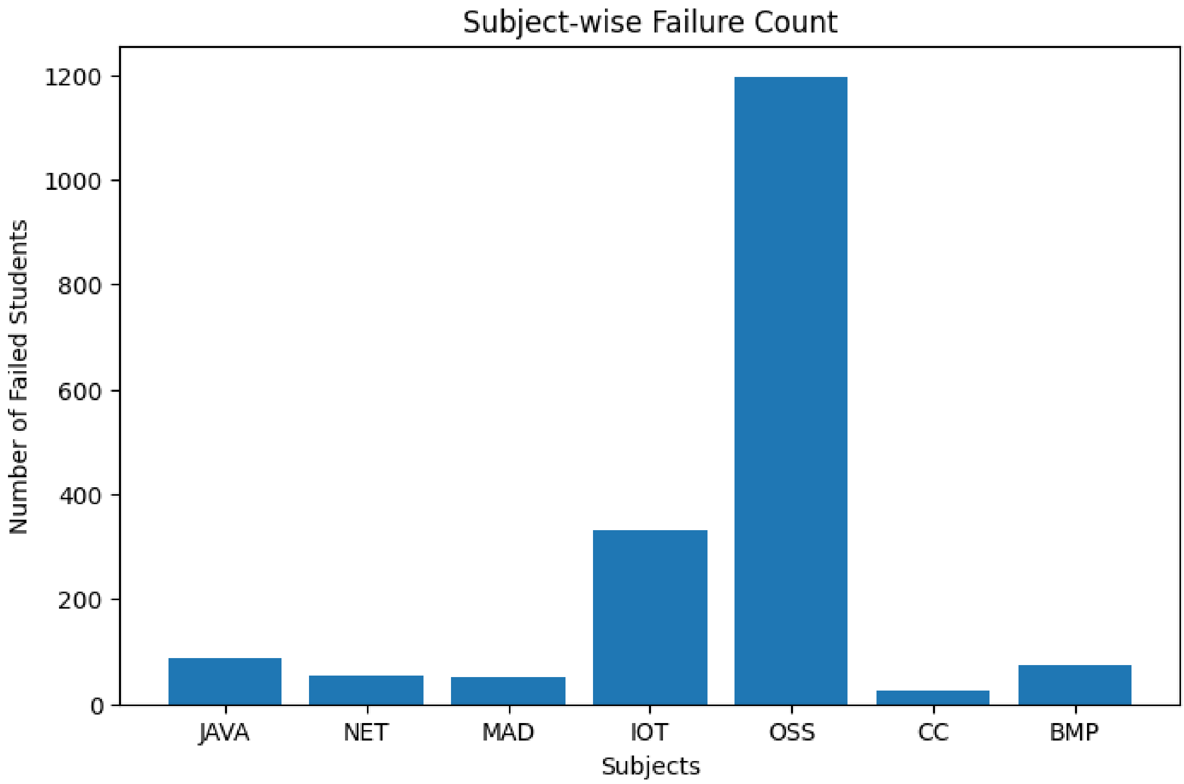


Interpretation:

- Higher PASS count → strong semester performance.
- Low FAIL/ATKT count → difficult subjects.

8.3 Failure Count Per Subject :

```python
failure_counts = {
    'JAVA': (df['JAVA_Total'] < 33).sum(),
    'NET': (df['NET_Total'] < 33).sum(),
    'MAD': (df['MAD_Total'] < 33).sum(),
    'IOT': (df['IOT_Total'] < 33).sum(),
    'OSS': (df['OSS_Total'] < 33).sum(),
    'CC': (df['CC_Total'] < 18).sum(),
    'BMP': (df['BMP_Total'] < 18).sum()
}

print(failure_counts)

# Plot bar chart for failure count
plt.figure(figsize=(8, 5))

plt.bar(
    failure_counts.keys(),
    failure_counts.values()
)

plt.title("Subject-wise Failure Count")
plt.xlabel("Subjects")
plt.ylabel("Number of Failed Students")

plt.show()
```

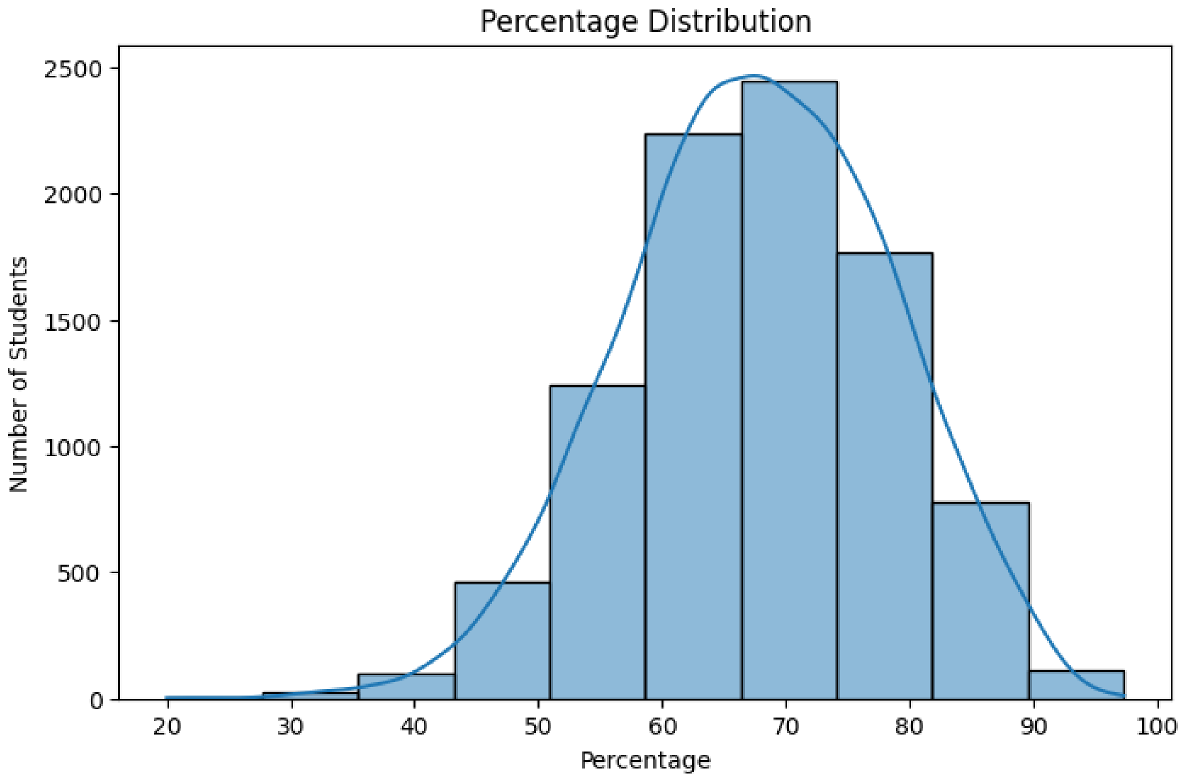


Interpretation:

- OSS has the highest failures (~1200), indicating it is the most difficult subject.
- IOT shows moderate failure count (~330), suggesting average difficulty.
- JAVA, NET, MAD, and BMP have relatively low failures, indicating better student performance.
- CC has the lowest failures (~30), showing strongest student performance.

8.4 Percentage Distribution :

```
plt.figure(figsize=(8,5))
sns.histplot(df['Percentage'], bins=10, kde=True)
plt.title("Percentage Distribution")
plt.xlabel("Percentage")
plt.ylabel("Number of Students")
plt.show()
```



Interpretation :

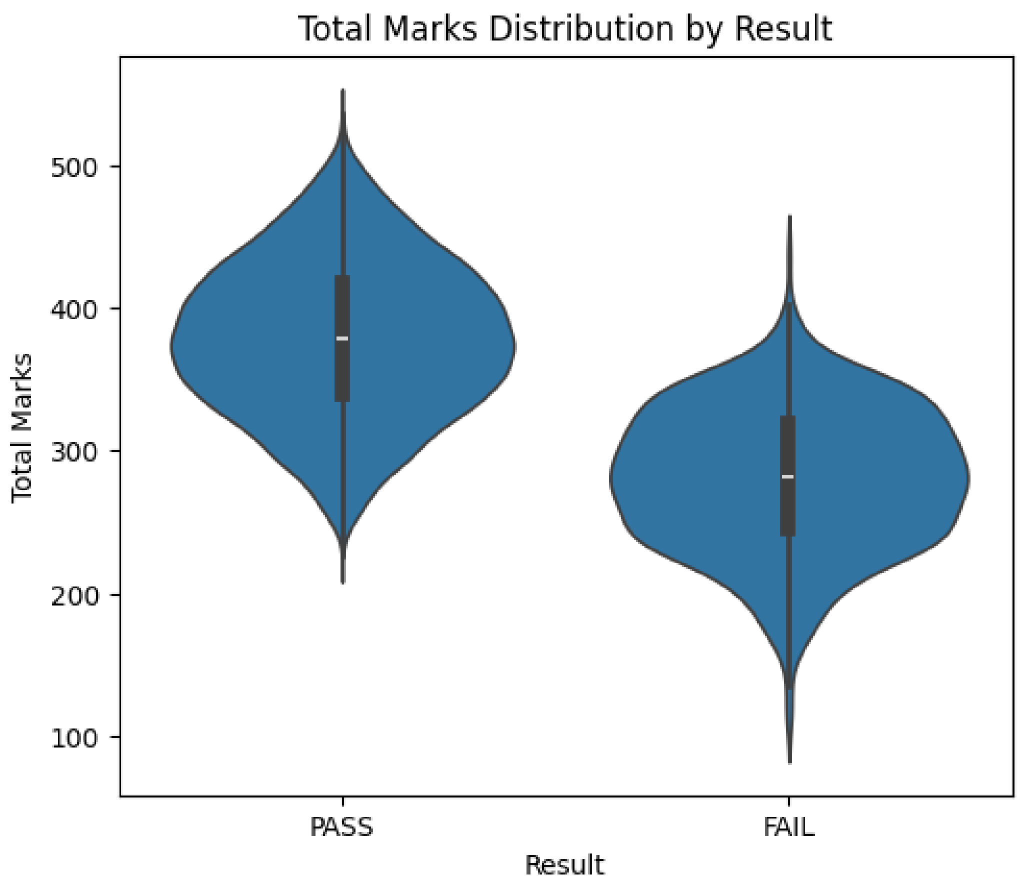
- Most students scored between 55% and 75%, indicating average to good overall performance.
- The distribution appears approximately bell-shaped (normal), showing balanced academic results.
- Very few students scored below 40% or above 90%, indicating limited extreme performance cases.

9. Bivariate Analysis

Bivariate Analysis involves examining the relationship between two variables to understand patterns, dependencies, and performance trends within the dataset. In this chapter, we analyse relationships between academic metrics such as total marks, percentage, result status, subject performance, and institutional comparison.

9.1 Distribution of Total Marks by Result :

```
sns.violinplot(x='Result', y='Total Marks', data=df)
plt.title("Distribution of Total Marks by Result")
plt.show()
```

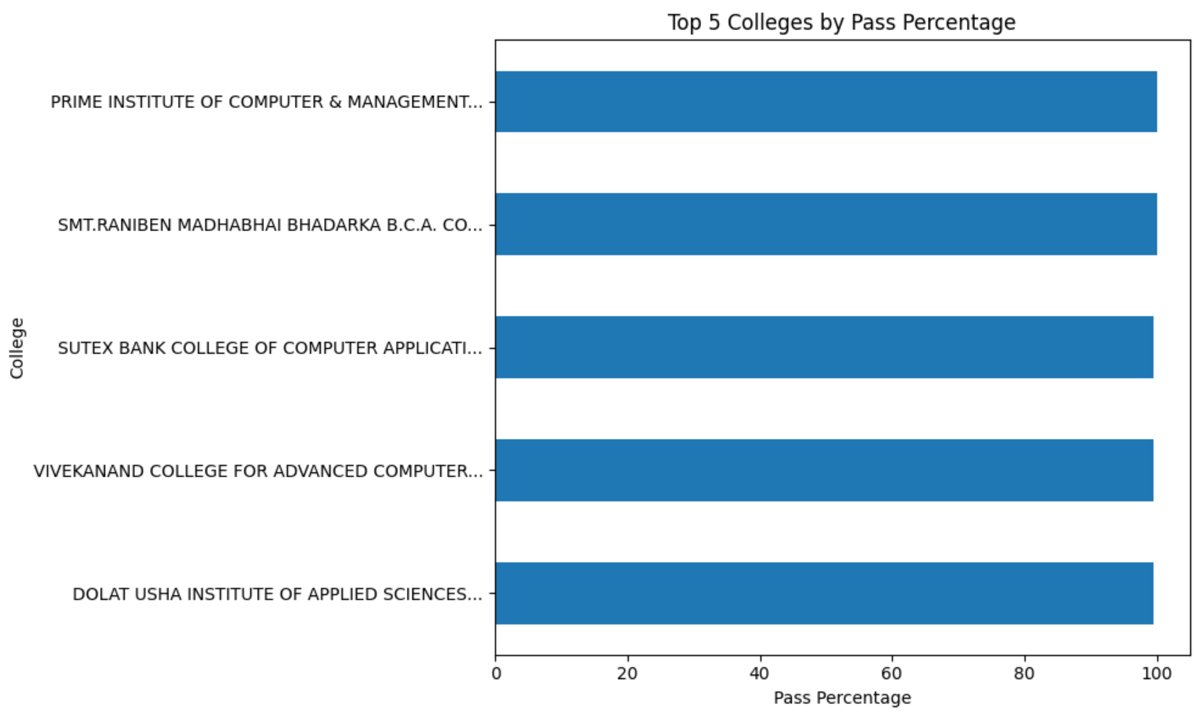


Interpretation:

The violin plot shows a clear distinction between PASS and NOT PASS students. Students who passed are concentrated in the higher total marks range, while non-passing students are clustered in the lower score region. The separation confirms that total marks strongly influence final result status.

9.2 Top 5 Colleges by Average Percentage :

```
college_avg = df.groupby('College Name')['Percentage'].mean().sort_values(ascending=False)

top5 = college_avg.head(5)

plt.figure(figsize=(8,5))
top5.sort_values().plot(kind='barh')
plt.title("Top 5 Colleges by Average Percentage")
plt.xlabel("Average Percentage")
plt.tight_layout()
plt.show()
```

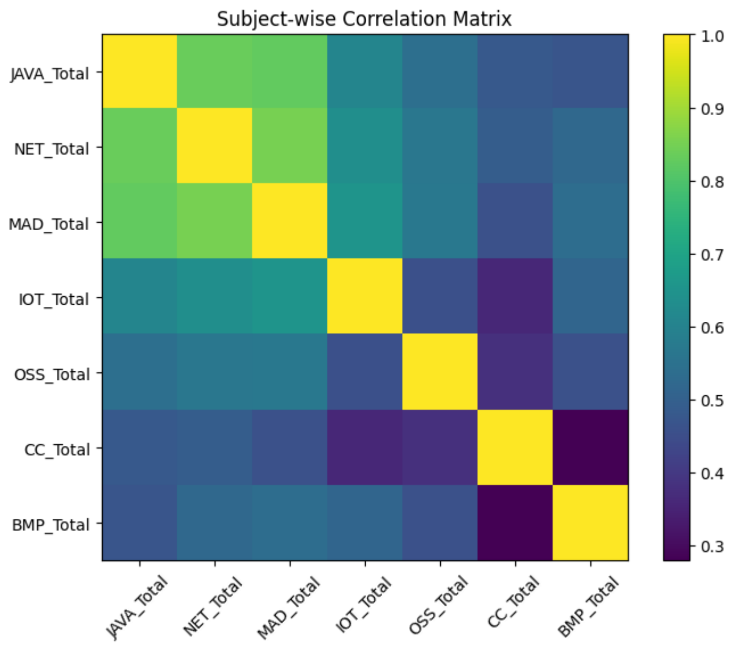


Interpretation:

The comparison reveals measurable differences in academic performance across institutions. The top-performing colleges demonstrate consistently higher average percentages, indicating stronger academic outcomes. This analysis highlights institutional variation under the same university framework.

9.3 Subject-wise Correlation Analysis :

```
Passing_percentage = {
    'JAVA': (df['JAVA_Total'] > 33).mean() * 100,
    'NET': (df['NET_Total'] > 33).mean() * 100,
    'MAD': (df['MAD_Total'] > 33).mean() * 100,
    'IOT': (df['IOT_Total'] > 33).mean() * 100,
    'OSS': (df['OSS_Total'] > 33).mean() * 100,
    'CC': (df['CC_Total'] > 18).mean() * 100,
    'BMP': (df['BMP_Total'] > 18).mean() * 100
}

subjects = list(Passing_percentage.keys())
percentages = list(Passing_percentage.values())

plt.figure(figsize=(8, 5))
plt.bar(subjects, percentages)

plt.ylim(50, 100)

plt.title("Subject-wise Passing Percentage")
plt.xlabel("Subjects")
plt.ylabel("Passing Percentage")

plt.show()
```

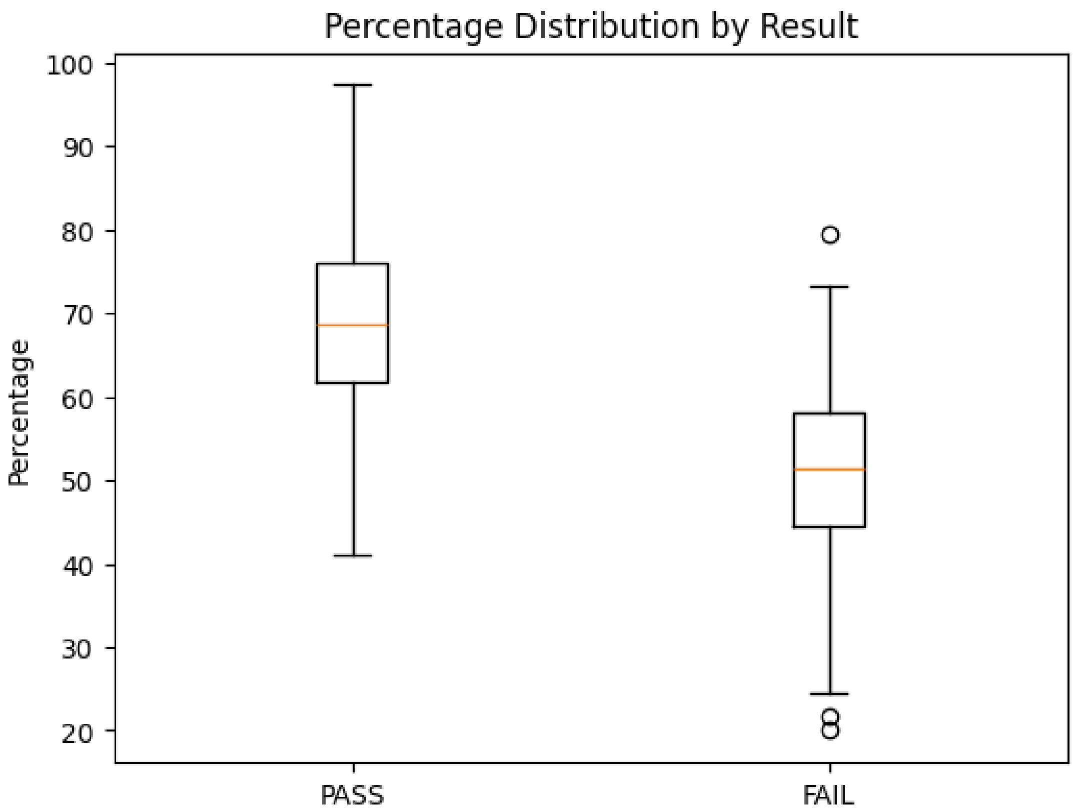


Interpretation:

The correlation heatmap indicates strong positive relationships among several theory-based subjects. High correlation values suggest that students who perform well in one subject tend to perform well in related subjects. Lower correlation values indicate relatively independent performance patterns.

9.4 Percentage Distribution by Result :

```python
pass_percent = df[df['Result'] == 'PASS']['Percentage']
fail_percent = df[df['Result'] != 'PASS']['Percentage']

plt.boxplot([pass_percent, fail_percent], labels=['PASS', 'NOT PASS'])
plt.title("Percentage Distribution by Result")
plt.ylabel("Percentage")
plt.show()
```



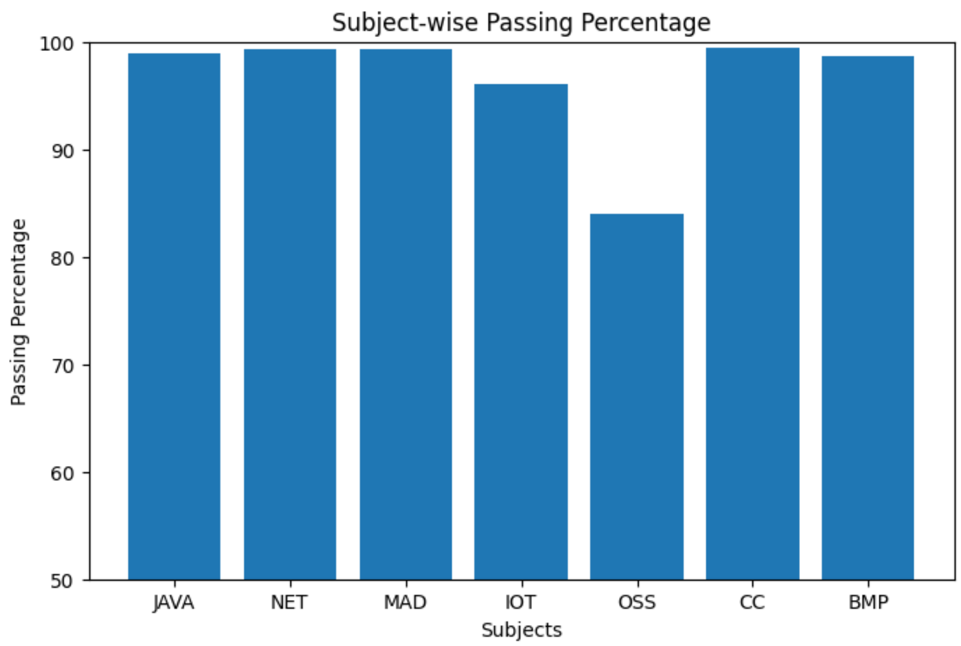
Interpretation:

The median percentage of PASS students is significantly higher than that of NOT PASS students. The boxplot clearly shows that percentage is a strong determinant of final academic outcome. The variability among failing students is greater, indicating inconsistent performance patterns.

9.5 Subject-wise Passing Percentage :

```
# Total number of students
total_students = len(df)

# Calculate passing percentage per subject
passing_percentage = {
    'AWD': (df['AWD_Total'] >= 33).sum() / total_students * 100,
    'WFS': (df['WFS_Total'] >= 33).sum() / total_students * 100,
    'NET': (df['.NET_Total'] >= 33).sum() / total_students * 100,
    'LOS': (df['LOS_Total'] >= 33).sum() / total_students * 100,
    'NT': (df['NT_Total'] >= 33).sum() / total_students * 100,
    'CC': (df['CC_Total'] >= 18).sum() / total_students * 100
}

plt.figure()

plt.bar(passing_percentage.keys(), passing_percentage.values())
```

```
plt.title("Subject-wise Passing Percentage")
plt.xlabel("Subjects")
plt.ylabel("Passing Percentage")

# Zoom into the important range
plt.ylim(90, 100)
plt.yticks(range(90, 101, 2))

plt.show()
```



Interpretation:

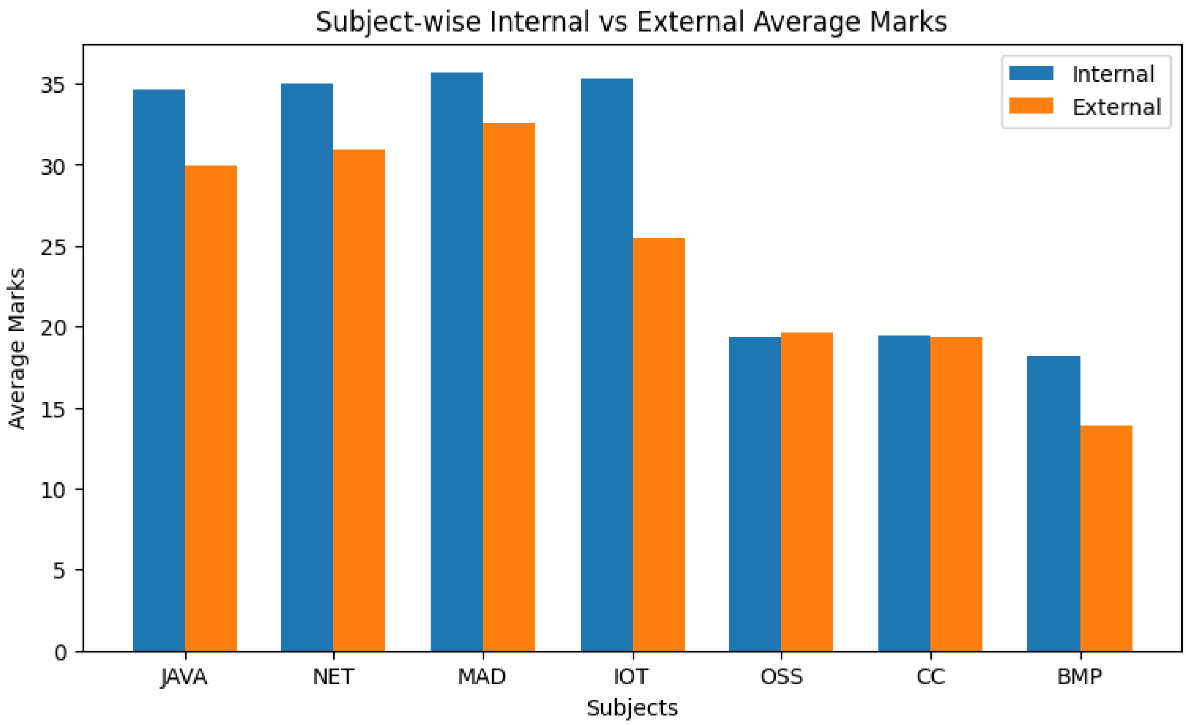
Most subjects demonstrate high passing percentages, indicating strong overall academic performance. However, comparatively lower passing rates in certain subjects highlight potential areas of academic difficulty. This analysis provides valuable insight for curriculum review and academic improvement strategies.

10.Multivariate Analysis

Multivariate analysis examines the relationship among three or more variables simultaneously to identify complex patterns within the dataset. In the context of student result analysis, multiple academic indicators such as subject marks, SGPA, percentage, and result status are analysed together to understand how different factors collectively influence student performance. This approach provides deeper insights compared to univariate or bivariate analysis by considering multiple dimensions of academic data at the same time.

10.1 Subject-wise Internal vs External Average Marks :

```
import numpy as np
import matplotlib.pyplot as plt

subjects = ["JAVA", "NET", "MAD", "IOT",
            "OSS", "CC", "BMP"]

internal_avg = [
    df['JAVA_Int'].mean(), df['NET_Int'].mean(),
    df['MAD_Int'].mean(), df['IOT_Int'].mean(),
    df['OSS_Int'].mean(), df['CC_Int'].mean(),
    df['BMP_Int'].mean()
]

external_avg = [
    df['JAVA_Ext'].mean(), df['NET_Ext'].mean(),
    df['MAD_Ext'].mean(), df['IOT_Ext'].mean(),
    df['OSS_Ext'].mean(), df['CC_Ext'].mean(),
    df['BMP_Ext'].mean()
]

x = np.arange(len(subjects))
width = 0.35

plt.figure(figsize=(9, 5))

plt.bar(x - width/2, internal_avg,
        width, label='Internal')
plt.bar(x + width/2, external_avg,
        width, label='External')

plt.xticks(x, subjects)
```

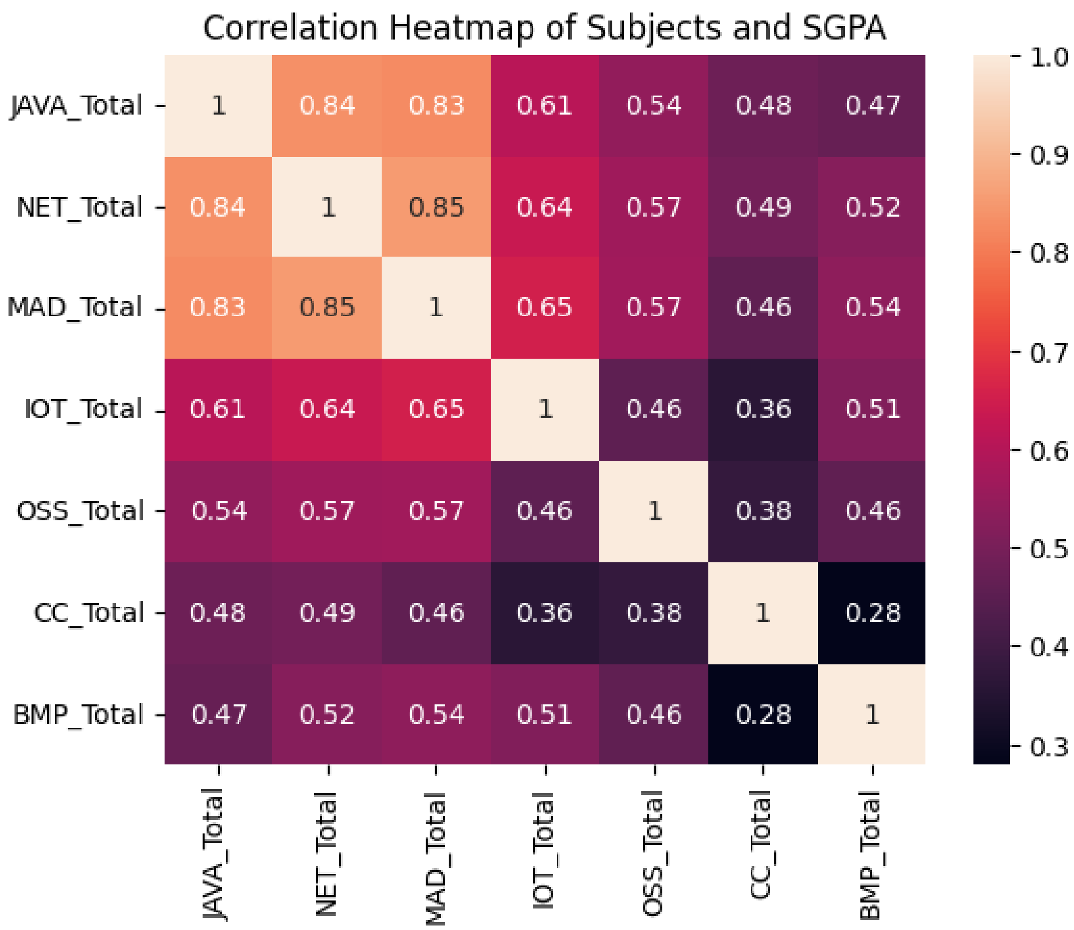


Interpretation :

This visualization compares the average internal and external marks for each subject. The results help identify whether internal assessments are more lenient compared to university examinations. Significant differences between the two can indicate variations in evaluation patterns.

10.2 Correlation Heatmap of Subjects and SGPA :

```
import seaborn as sns

cols = ["JAVA_Total", "NET_Total", "MAD_Total",
        "IOT_Total", "OSS_Total", "CC_Total",
        "BMP_Total"]

plt.figure()

sns.heatmap(
    df[cols].corr(),
    annot=True
)

plt.title("Correlation Heatmap of Subjects and SGPA")

plt.show()
```



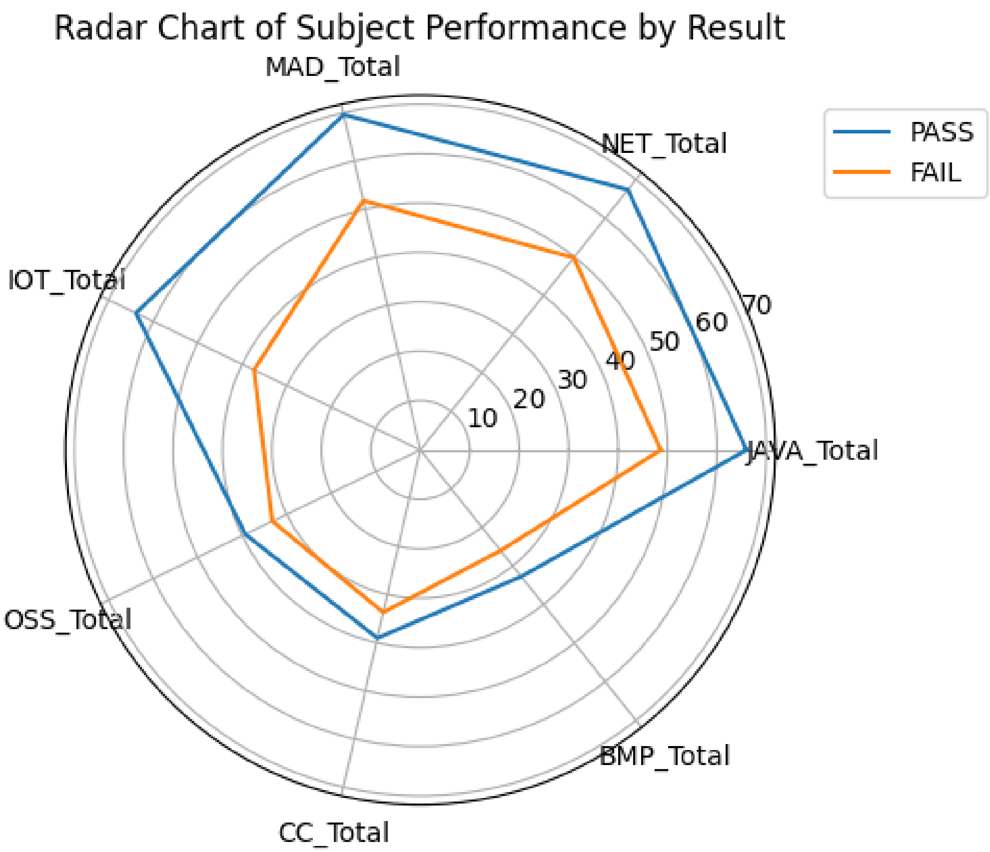
Interpretation :

The heatmap illustrates the strength of relationships between subject marks, SGPA, and percentage. Higher positive correlations suggest that strong performance in those subjects contributes significantly to overall academic achievement.

10.3 Radar Chart of Subject Performance by Result :

```
import numpy as np
import matplotlib.pyplot as plt

subjects = ["JAVA_Total", "NET_Total", "MAD_Total",
            "IOT_Total", "OSS_Total", "CC_Total",
            "BMP_Total"]

pass_avg = df[df['Result'] == 'PASS'][subjects] \
            .mean().values

fail_avg = df[df['Result'] != 'PASS'][subjects] \
            .mean().values

angles = np.linspace(0, 2*np.pi,
                     len(subjects),
                     endpoint=False)

pass_avg = np.concatenate(
    (pass_avg, [pass_avg[0]])
)
fail_avg = np.concatenate(
    (fail_avg, [fail_avg[0]])
)
angles = np.concatenate(
    (angles, [angles[0]])
)

fig = plt.figure()
ax = fig.add_subplot(111, polar=True)
```

```
ax.plot(angles, pass_avg, label='PASS')
ax.plot(angles, fail_avg, label='FAIL')

ax.set_xticks(angles[:-1])
ax.set_xticklabels(subjects)

plt.title("Radar Chart of Subject Performance by Result")

plt.legend(loc='upper left',
           bbox_to_anchor=(1.05, 1))

plt.show()
```

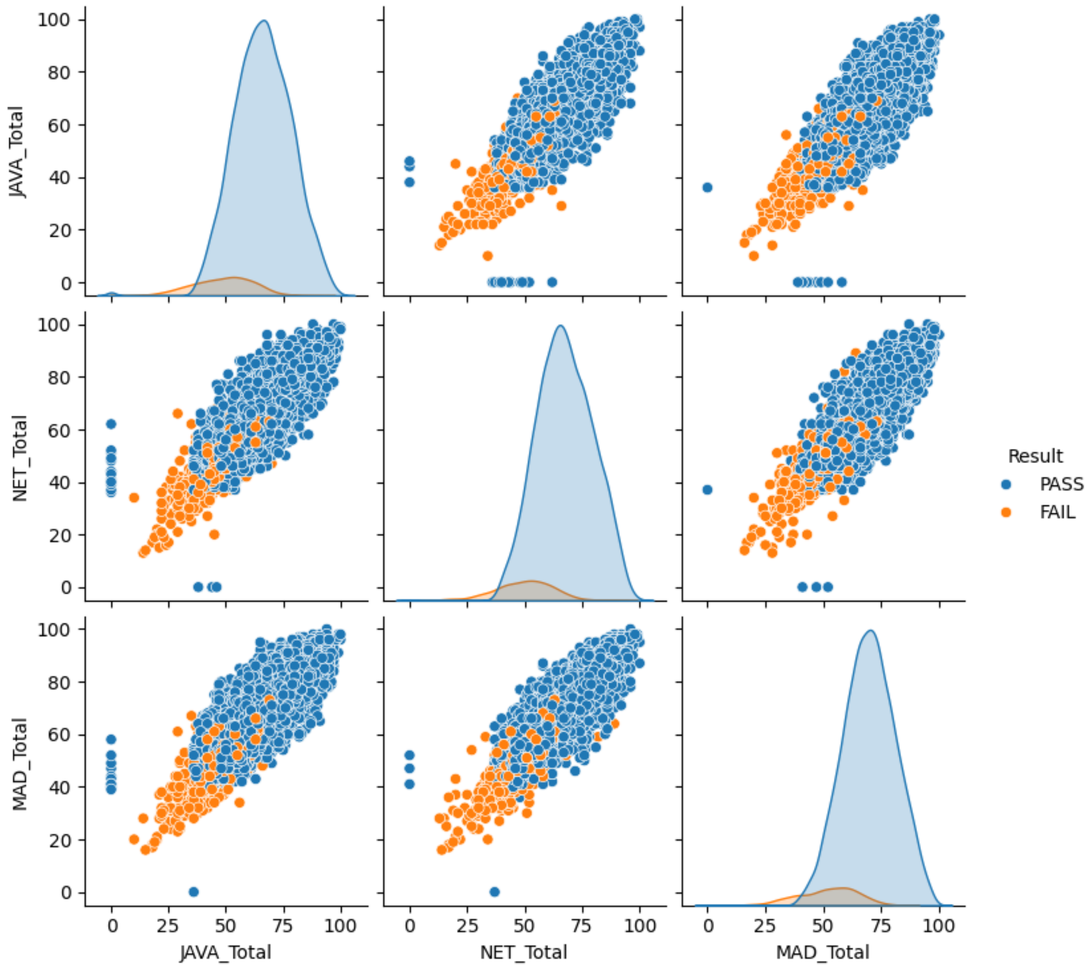


Interpretation :

The radar chart visualizes differences in subject-wise performance between passing and failing students. Subjects where failing students score significantly lower indicate areas that may contribute most to academic failure.

10.4 Pair Plot (Scatter Matrix) of Core Subjects :

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.pairplot(
    df[["JAVA_Total", "NET_Total",
        "MAD_Total", "Result"]],
    hue='Result'
)

plt.show()
```



Interpretation

The pair plot provides a comprehensive view of relationships between core subjects. The colour differentiation highlights clusters of passing and failing students, helping identify subject performance patterns associated with academic success.

11. Data Visualization Using Sweetviz

Data visualization plays a crucial role in understanding patterns, trends, and relationships within a dataset. Instead of manually creating multiple charts, the Sweetviz library was used to generate an automated exploratory data analysis (EDA) report. Sweetviz provides an interactive visualization dashboard that summarizes dataset characteristics including distributions, correlations, missing values, and variable relationships.

The generated report provides a comprehensive overview of student performance data in an easy-to-understand visual format.

11.1 Sweetviz Library Overview :

Sweetviz is a Python library designed for automated exploratory data analysis. It generates an interactive HTML report containing statistical summaries and visualizations for all variables in a dataset.

Key features include:

- Variable distribution visualization
- Target variable analysis
- Correlation analysis
- Missing value detection
- Feature comparison
- Automatic chart generation

This tool significantly reduces the effort required to manually create multiple charts for dataset exploration.

```
import sweetviz as sv


my_report = sv.analyze(df)
my_report.show_html()
```

19/03/2026, 11:33 SWEETVIZ_REPORT.html

Sweetviz 2.3.1
Get updates, docs & report issues here
Created & maintained by Francois Bertrand
Graphic design by Jean-Francois Hains

DataFrame — NO COMPARISON TARGET

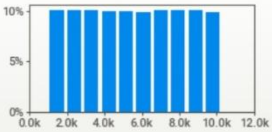
9157	ROWS
0	DUPLICATES
6.5 MB	RAM
45	FEATURES
5	CATEGORICAL
39	NUMERICAL
1	TEXT

ASSOCIATIONS

DataFrame

1 Seat Number

VALUES:	9,157 (100%)	MAX	10,249	RANGE	9,248
MISSING:	---	95%	9,775	IQR	4,627
DISTINCT:	9,157 (100%)	Q3	7,929	STD	2,670
ZEROES:	---	AVG	5,619	VAR	7.1M
		MEDIAN	5,615	KURT.	-1.20
		Q1	3,302	SKEW	9.17e-4
		5%	1,464	SUM	51.5M
		MIN	1,001		



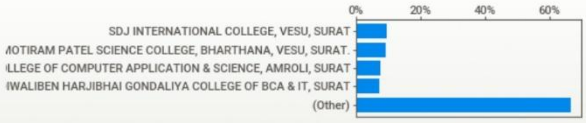
2 Name

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 9,145 (>99%)

2	<1%	PATEL KHUSHI MANOJBHAI
2	<1%	RADADIYA PRINCE ASHOKBHAI
2	<1%	PATEL PRINCE NILESHBHAI
2	<1%	PATIL HARSHAL DINESH
2	<1%	PATIL SUMIT SANJAY
2	<1%	PATEL YASHKUMAR DINESHBHAI
2	<1%	PATEL KRISHKUMAR YOGESHBHAI
9,143	>99%	(Other)

3 College Name

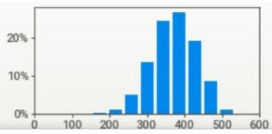
VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 36 (<1%)



4 Total Marks

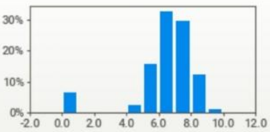
VALUES: 9,157 (100%) MAX 535 RANGE 425

MISSING:	---	95%	467	IQR	83.0
DISTINCT:	345 (4%)	Q3	415	STD	59.3
ZEROES:	---	MEDIAN	373	VAR	3,516
		AVG	372	KURT.	-0.121
		Q1	332	SKEW	-0.198
		5%	273	SUM	3.4M
		MIN	110		



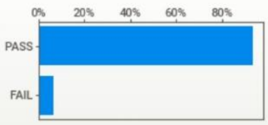
5 SGPA

VALUES:	9,157 (100%)	MAX	10.0	RANGE	10.0
MISSING:	---	95%	8.4	IQR	1.55
DISTINCT:	65 (<1%)	Q3	7.5	STD	1.95
ZEROES:	602 (7%)	MEDIAN	6.7	VAR	3.80
		AVG	6.4	KURT.	4.89
		Q1	6.0	SKEW	-2.15
		5%	0.0	SUM	58,608
		MIN	0.0		



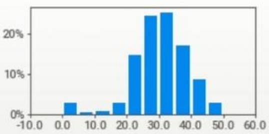
6 Result

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 2 (<1%)



7 JAVA_Ext

VALUES:	9,157 (100%)	MAX	50.0	RANGE	50.0
MISSING:	---	95%	43.0	IQR	10.0
DISTINCT:	49 (<1%)	Q3	35.0	STD	8.61
ZEROES:	239 (3%)	MEDIAN	30.0	VAR	74.1
		AVG	29.9	KURT.	2.35
		Q1	25.0	SKEW	-0.938
		5%	18.0	SUM	274k
		MIN	0.0		



8 JAVA_Int

VALUES:	9,157 (100%)	MAX	50.0	RANGE	50.0
MISSING:	---	95%	46.0	IQR	10.0
DISTINCT:	47 (<1%)	Q3	40.0	STD	6.96
ZEROES:	1 (<1%)	MEDIAN	35.0	VAR	48.5
		AVG	34.6	KURT.	-0.126
		Q1	30.0	SKEW	-0.280
		5%	22.0		

19/03/2026, 11:33 SWEETVIZ_REPORT.html

MIN 0.0 SUM 317k

9 JAVA_Total

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 87 (<1%)
ZEROES: 15 (<1%)

MAX	100	RANGE	100
95%	86	IQR	18.0
Q3	74	STD	13.2
MEDIAN	65	VAR	173
AVG	65		
Q1	56	KURT.	0.646
5%	44	SKEW	-0.281
MIN	0	SUM	594k

10 JAVA_Th_Ext

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 26 (<1%)
ZEROES: 14 (<1%)

MAX	25.0	RANGE	25.0
95%	21.0	IQR	6.00
Q3	16.0	STD	4.53
AVG	13.2	VAR	20.5
MEDIAN	13.0		
Q1	10.0	KURT.	-0.336
5%	6.0	SKEW	0.013
MIN	0.0	SUM	121k

11 JAVA_Th_Int

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 25 (<1%)
ZEROES: 1 (<1%)

MAX	25.0	RANGE	25.0
95%	23.0	IQR	5.00
Q3	20.0	STD	3.81
AVG	17.1	VAR	14.5
MEDIAN	17.0		
Q1	15.0	KURT.	-0.472
5%	10.0	SKEW	-0.213
MIN	0.0	SUM	156k

12 JAVA_Pr_Ext

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 25 (<1%)
ZEROES: 73 (<1%)

MAX	25.0	RANGE	25.0
95%	24.0	IQR	6.00
Q3	20.0	STD	4.78
MEDIAN	18.0	VAR	22.9
AVG	17.1		
Q1	14.0	KURT.	0.583
5%	10.0	SKEW	-0.607
MIN	0.0	SUM	156k

13 JAVA_Pr_Int

VALUES: 9,157 (100%) MAX 25.0 RANGE 25.0

file:///D:/VNSGU_PDA2/Data-Analytics-using-Python-Minor-Project/Notebooks/SWEETVIZ_REPORT.html 3/11

19/03/2026, 11:33 SWEETVIZ_REPORT.html

MISSING: ---
DISTINCT: 26 (<1%)
ZEROES: 9 (<1%)

95%	24.0	IQR	5.00
Q3	20.0	STD	4.22
MEDIAN	18.0	VAR	17.8
AVG	17.5		
Q1	15.0	KURT.	-0.210
5%	10.0	SKEW	-0.406
MIN	0.0	SUM	161k

14 NET_Ext

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 50 (<1%)
ZEROES: 164 (2%)

MAX	50.0	RANGE	50.0
95%	43.0	IQR	10.0
Q3	36.0	STD	8.20
MEDIAN	31.0	VAR	67.2
AVG	30.9		
Q1	26.0	KURT.	2.35
5%	19.0	SKEW	-0.877
MIN	0.0	SUM	283k

15 NET_Int

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 45 (<1%)
ZEROES: ---

MAX	50.0	RANGE	46.0
95%	46.0	IQR	9.00
Q3	40.0	STD	6.76
AVG	35.1	VAR	45.6
MEDIAN	35.0		
Q1	31.0	KURT.	-0.237
5%	23.0	SKEW	-0.229
MIN	4.0	SUM	321k

16 NET_Total

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 87 (<1%)
ZEROES: 3 (<1%)

MAX	100	RANGE	100
95%	87	IQR	18.0
Q3	75	STD	12.9
AVG	66	VAR	166
MEDIAN	66		
Q1	57	KURT.	0.019
5%	45	SKEW	-0.153
MIN	0	SUM	607k

17 NET_Th_Ext

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 26 (<1%)
ZEROES: 16 (<1%)

MAX	25.0	RANGE	25.0
95%	21.0	IQR	6.00
Q3	17.0	STD	4.41
MEDIAN	14.0	VAR	19.5
AVG	13.8		
Q1	11.0	KURT.	-0.282
5%	6.0	SKEW	-0.134

file:///D:/VNSGU_PDA2/Data-Analytics-using-Python-Minor-Project/Notebooks/SWEETVIZ_REPORT.html 4/11

19/03/2026, 11:33 SWEETVIZ_REPORT.html

MIN 0.0 SUM 127k

18 NET_Th_Int

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 22 (<1%)
ZEROES: ---

MAX 25.0 · 95% 23.0 · Q3 20.0 · MEDIAN 18.0 · AVG 17.4 · Q1 15.0 · 5% 11.0 · MIN 4.0

RANGE 21.0 · IQR 5.00 · STD 3.70 · VAR 13.7 · KURT. -0.490 · SKEW -0.229 · SUM 159k

19 NET_Pr_Ext

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 25 (<1%)
ZEROES: 74 (<1%)

MAX 25.0 · 95% 24.0 · Q3 20.0 · MEDIAN 18.0 · AVG 17.3 · Q1 15.0 · 5% 10.0 · MIN 0.0

RANGE 25.0 · IQR 5.00 · STD 4.42 · VAR 19.5 · KURT. 0.979 · SKEW -0.528 · SUM 159k

20 NET_Pr_Int

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 25 (<1%)
ZEROES: 7 (<1%)

MAX 25.0 · 95% 24.0 · Q3 21.0 · MEDIAN 18.0 · AVG 17.7 · Q1 15.0 · 5% 10.0 · MIN 0.0

RANGE 25.0 · IQR 6.00 · STD 4.20 · VAR 17.7 · KURT. -0.346 · SKEW -0.369 · SUM 162k

21 MAD_Ext

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 50 (<1%)
ZEROES: 90 (<1%)

MAX 50.0 · 95% 44.0 · Q3 38.0 · MEDIAN 33.0 · AVG 32.6 · Q1 28.0 · 5% 21.0 · MIN 0.0

RANGE 50.0 · IQR 10.0 · STD 7.45 · VAR 55.5 · KURT. 2.94 · SKEW -0.949 · SUM 299k

22 MAD_Int

VALUES: 9,157 (100%) · MAX 50.0 · RANGE 40.0

file:///D:/VNSGU_PDA2/Data-Analytics-using-Python-Minor-Project/Notebooks/SWEETVIZ_REPORT.html 5/11

19/03/2026, 11:33 SWEETVIZ_REPORT.html

MISSING: ---
DISTINCT: 40 (<1%)
ZEROES: ---

MEDIAN 36.0 · AVG 35.7 · Q1 31.0 · 5% 24.0 · MIN 10.0

VAR 43.4 · KURT. -0.296 · SKEW -0.288 · SUM 327k

23 MAD_Total

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 85 (<1%)
ZEROES: 1 (<1%)

MAX 100 · 95% 88 · Q3 77 · MEDIAN 69 · AVG 68 · Q1 61 · 5% 48 · MIN 0

RANGE 100 · IQR 16.0 · STD 12.2 · VAR 148 · KURT. 0.236 · SKEW -0.291 · SUM 627k

24 MAD_Th_Ext

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 26 (<1%)
ZEROES: 21 (<1%)

MAX 25.0 · 95% 22.0 · Q3 19.0 · MEDIAN 16.0 · AVG 15.4 · Q1 12.0 · 5% 8.0 · MIN 0.0

RANGE 25.0 · IQR 7.00 · STD 4.34 · VAR 18.8 · KURT. -0.092 · SKEW -0.330 · SUM 141k

25 MAD_Th_Int

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 22 (<1%)
ZEROES: ---

MAX 25.0 · 95% 23.0 · Q3 21.0 · MEDIAN 18.0 · AVG 18.0 · Q1 16.0 · 5% 11.0 · MIN 2.0

RANGE 23.0 · IQR 5.00 · STD 3.62 · VAR 13.1 · KURT. -0.232 · SKEW -0.404 · SUM 164k

26 MAD_Pr_Ext

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 25 (<1%)
ZEROES: 75 (<1%)

MAX 25.0 · 95% 24.0 · Q3 20.0 · MEDIAN 18.0 · AVG 17.4 · Q1 15.0 · 5% 10.0

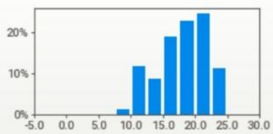
RANGE 25.0 · IQR 5.00 · STD 4.14 · VAR 17.1 · KURT. 1.98 · SKEW -0.811 · SUM 159k

file:///D:/VNSGU_PDA2/Data-Analytics-using-Python-Minor-Project/Notebooks/SWEETVIZ_REPORT.html 6/11

		MIN	0.0

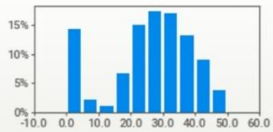
27 MAD_Pr_Int

VALUES:	9,157 (100%)	MAX	25.0	RANGE	25.0
MISSING:	---	95%	24.0	IQR	6.00
		Q3	21.0	STD	4.03
DISTINCT:	22 (<1%)	MEDIAN	18.0	VAR	16.2
		AVG	17.7		
ZEROES:	4 (<1%)	Q1	15.0	KURT.	-0.327
		5%	10.0	SKEW	-0.406
		MIN	0.0	SUM	162k



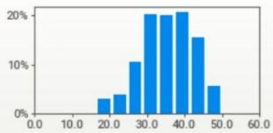
28 IOT_Ext

VALUES:	9,157 (100%)	MAX	50.0	RANGE	50.0
MISSING:	---	95%	43.0	IQR	15.0
		Q3	35.0	STD	13.2
DISTINCT:	49 (<1%)	MEDIAN	28.0	VAR	173
		AVG	25.4		
ZEROES:	1,182 (13%)	Q1	20.0	KURT.	-0.381
		5%	0.0	SKEW	-0.652
		MIN	0.0	SUM	233k



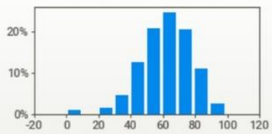
29 IOT_Int

VALUES:	9,157 (100%)	MAX	50.0	RANGE	42.0
MISSING:	---	95%	46.0	IQR	11.0
		Q3	41.0	STD	7.00
DISTINCT:	36 (<1%)	MEDIAN	36.0	VAR	49.0
		AVG	35.3		
ZEROES:	---	Q1	30.0	KURT.	-0.402
		5%	23.0	SKEW	-0.364
		MIN	8.0	SUM	324k



30 IOT_Total

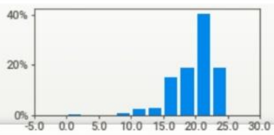
VALUES:	9,157 (100%)	MAX	99.0	RANGE	99.0
MISSING:	---	95%	86.0	IQR	22.0
		Q3	74.0	STD	16.2
DISTINCT:	84 (<1%)	MEDIAN	63.0	VAR	261
		AVG	62.1		
ZEROES:	105 (1%)	Q1	52.0	KURT.	1.31
		5%	36.0	SKEW	-0.705
		MIN	0.0	SUM	569k



31 OSS_Ext

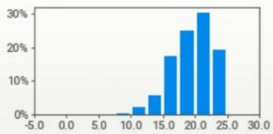
VALUES:	9,157 (100%)	MAX	25.0	RANGE	25.0

MISSING:	---	95% / Q3	24.0 / 22.0	IQR / STD	4.00 / 3.43
DISTINCT:	19 (<1%)	MEDIAN	20.0	VAR	11.8
		AVG	19.6		
ZEROES:	34 (<1%)	Q1	18.0	KURT.	3.89
		5%	14.0	SKEW	-1.31
		MIN	0.0	SUM	179k



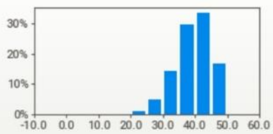
32 OSS_Int

VALUES:	9,157 (100%)	MAX	25.0	RANGE	25.0
MISSING:	---	95%	24.0	IQR	5.00
		Q3	22.0	STD	3.29
DISTINCT:	23 (<1%)	AVG	19.3	VAR	10.8
		MEDIAN	19.0		
ZEROES:	7 (<1%)	Q1	17.0	KURT.	0.837
		5%	13.0	SKEW	-0.641
		MIN	0.0	SUM	177k



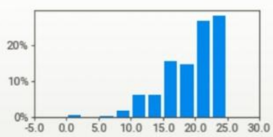
33 OSS_Total

VALUES:	9,157 (100%)	MAX	50.0	RANGE	50.0
MISSING:	---	95%	47.0	IQR	7.00
		Q3	43.0	STD	5.77
DISTINCT:	44 (<1%)	MEDIAN	40.0	VAR	33.3
		AVG	38.9		
ZEROES:	7 (<1%)	Q1	36.0	KURT.	2.65
		5%	29.0	SKEW	-0.919
		MIN	0.0	SUM	356k



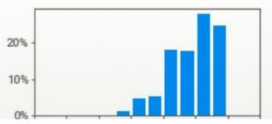
34 CC_Ext

VALUES:	9,157 (100%)	MAX	25.0	RANGE	25.0
MISSING:	---	95%	25.0	IQR	6.00
		Q3	23.0	STD	4.46
DISTINCT:	22 (<1%)	MEDIAN	20.0	VAR	19.9
		AVG	19.3		
ZEROES:	50 (<1%)	Q1	17.0	KURT.	1.03
		5%	11.0	SKEW	-0.922
		MIN	0.0	SUM	177k



35 CC_Int

VALUES:	9,157 (100%)	MAX	25.0	RANGE	25.0
MISSING:	---	95%	25.0	IQR	5.00
		Q3	22.0	STD	3.92
DISTINCT:	19 (<1%)	MEDIAN	20.0	VAR	15.4
		AVG	19.4		
ZEROES:	1 (<1%)	Q1	17.0	KURT.	-0.218
		5%	12.0	SKEW	-0.578
		MIN	0.0	SUM	178k



19/03/2026, 11:33 SWEETVIZ_REPORT.html

36 CC_Total

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 41 (<1%)
ZEROES: 4 (<1%)

MAX	50.0	RANGE	50.0
95%	50.0	IQR	10.0
Q3	44.0	STD	7.43
MEDIAN	40.0	VAR	55.3
AVG	38.7		
Q1	34.0	KURT.	-0.077
5%	25.0	SKEW	-0.538
MIN	0.0	SUM	354k

37 BMP_Ext

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 24 (<1%)
ZEROES: 400 (4%)

MAX	25.0	RANGE	25.0
95%	21.0	IQR	6.00
Q3	17.0	STD	4.84
MEDIAN	14.0	VAR	23.4
AVG	13.9		
Q1	11.0	KURT.	1.21
5%	5.0	SKEW	-0.656
MIN	0.0	SUM	127k

38 BMP_Int

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 19 (<1%)
ZEROES: 1 (<1%)

MAX	25.0	RANGE	25.0
95%	24.0	IQR	5.00
Q3	21.0	STD	3.82
MEDIAN	19.0	VAR	14.6
AVG	18.2		
Q1	16.0	KURT.	-0.567
5%	11.0	SKEW	-0.393
MIN	0.0	SUM	167k

39 BMP_Total

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 43 (<1%)
ZEROES: 28 (<1%)

MAX	50.0	RANGE	50.0
95%	43.0	IQR	9.00
Q3	37.0	STD	6.54
MEDIAN	33.0	VAR	42.8
AVG	32.3		
Q1	28.0	KURT.	1.10
5%	22.0	SKEW	-0.416
MIN	0.0	SUM	296k

40 Course_Name

VALUES: 9,157 (100%)
MISSING: ---

file:///D:/VNSGU_PDA2/Data-Analytics-using-Python-Minor-Project/Notebooks/SWEETVIZ_REPORT.html 9/11

19/03/2026, 11:33 SWEETVIZ_REPORT.html

DISTINCT: 33 (<1%)

41 Percentage

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 345 (4%)
ZEROES: ---

MAX	97.3	RANGE	77.3
95%	84.9	IQR	15.1
Q3	75.5	STD	10.8
MEDIAN	67.8	VAR	116
AVG	67.7		
Q1	60.4	KURT.	-0.121
5%	49.6	SKEW	-0.198
MIN	20.0	SUM	620k

42 Failed

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 2 (<1%)

43 Total_Internal

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 176 (2%)
ZEROES: ---

MAX	275	RANGE	199
95%	245	IQR	43.0
Q3	220	STD	31.2
MEDIAN	200	VAR	971
AVG	198		
Q1	177	KURT.	-0.307
5%	143	SKEW	-0.309
MIN	76	SUM	1.8M

44 Total_External

VALUES: 9,157 (100%)
MISSING: ---
DISTINCT: 226 (2%)
ZEROES: 2 (<1%)

MAX	266	RANGE	266
95%	227	IQR	48.0
Q3	197	STD	35.9
MEDIAN	174	VAR	1,288
AVG	172		
Q1	149	KURT.	0.404
5%	110	SKEW	-0.452
MIN	0	SUM	1.6M

file:///D:/VNSGU_PDA2/Data-Analytics-using-Python-Minor-Project/Notebooks/SWEETVIZ_REPORT.html 10/11

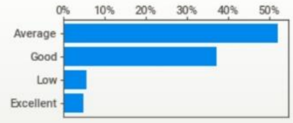
19/03/2026, 11:33 SWEETVIZ_REPORT.html

-50 0 50 100 150 200 250 300

45 Performance_Level

VALUES: 9,157 (100%)
MISSING: ---

DISTINCT: 4 (<1%)



file:///D:/VNSGU_PDA2/Data-Analytics-using-Python-Minor-Project/Notebooks/SWEETVIZ_REPORT.html 11/11

12.Machine Learning Implementation

In this chapter, the Python implementation used for student performance clustering is presented.
The code performs the following steps:

- Import required libraries
- Load cleaned dataset
- Check missing values
- Standardize features
- Apply K-Means Clustering
- Determine optimal clusters using Elbow Method
- Assign clusters to students
- Label clusters based on performance
- Visualize clusters using PCA
- Evaluate clustering using Silhouette Score
- Save trained model and results

12.1 Import Required Libraries :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
```

12.2 Load Dataset :

```
file_path = "D:\\VNSGU_PDA2\\Data-Analytics-using-\
Python-Minor-Project\\Data\\raw\\VNSGU_SEM4_\
All_Student_Results.csv"

df = pd.read_csv(file_path)
```

12.3 Select Features for Clustering :

```
features = ["JAVA_Total", "NET_Total", "MAD_Total",
            "IOT_Total", "OSS_Total", "CC_Total",
            "BMP_Total"]

X = df[features]
```

12.4 Check Missing Values :

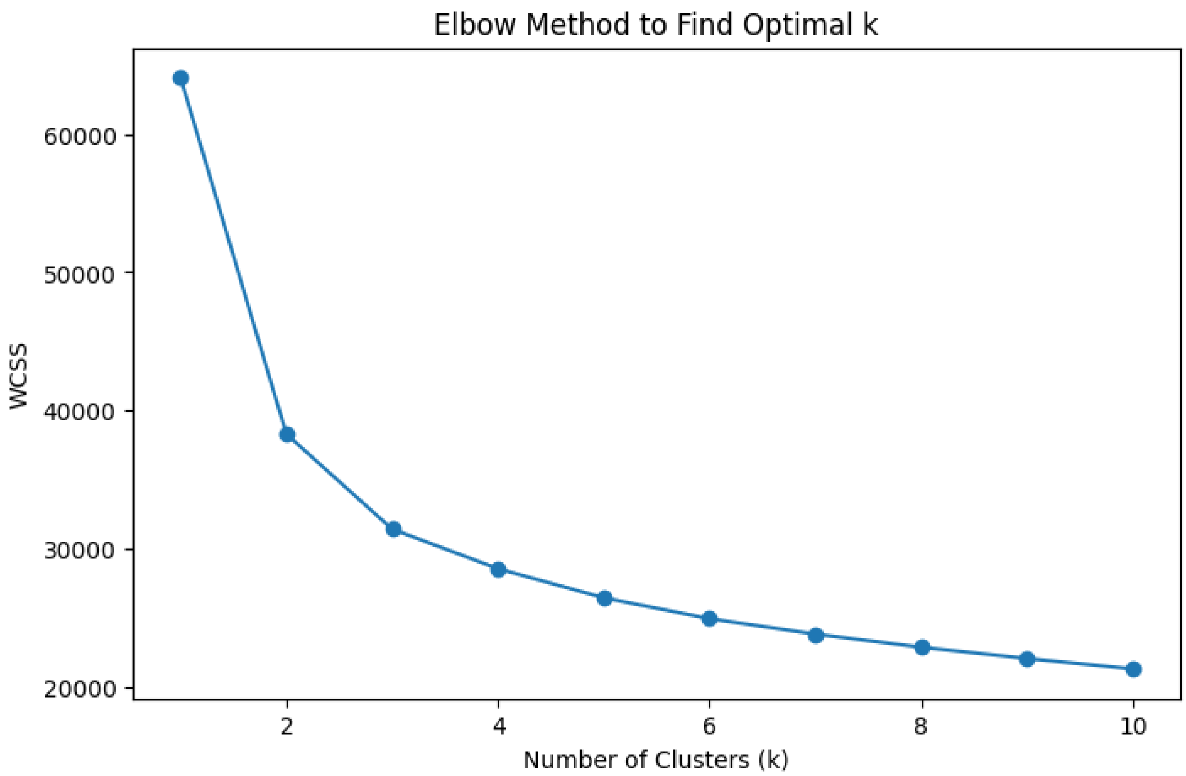
```
missing_values = df[features].isnull().sum()

print(missing_values)
```

12.5 Standardize the Data :

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

12.6 Determine Optimal Clusters using Elbow Method :

```
inertia = []
k_range = range(1, 10)

for k in k_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertia.append(km.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(k_range, inertia, marker='o')

plt.title("Elbow Method for Optimal k")
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")

plt.show()
```



12.7 Apply K-Means Clustering :

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

df['Cluster'] = kmeans.fit_predict(X_scaled)
```

12.8 Count Students in Each Cluster :

```
cluster_counts = df['Cluster'].value_counts()

print(cluster_counts)
```

12.9 Cluster-wise Performance Summary :

```
cluster_summary = df.groupby("Cluster")[features] \
                  .mean()

print(cluster_summary)
```

12.10 Assign Performance Labels :

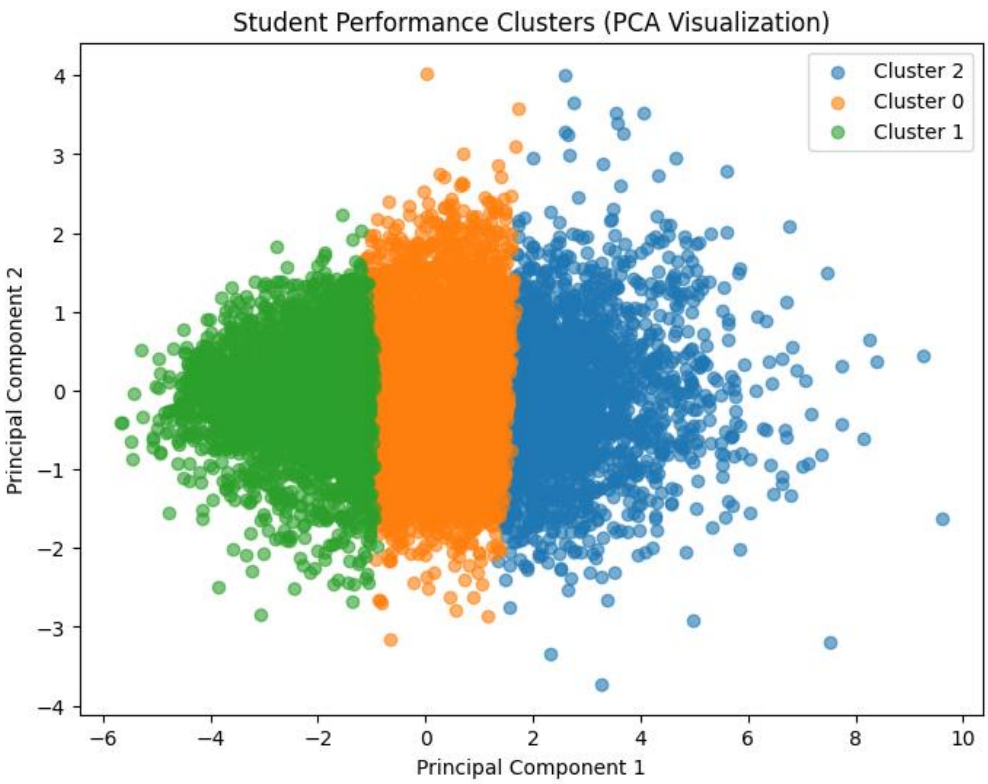
```
cluster_labels = {
    0: "Low Performance",
    1: "Average Performance",
    2: "High Performance"
}

df['Performance'] = df['Cluster'].map(cluster_labels)
```

12.11 PCA Visualization :

```
pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

df['PCA1'] = X_pca[:, 0]
df['PCA2'] = X_pca[:, 1]
```

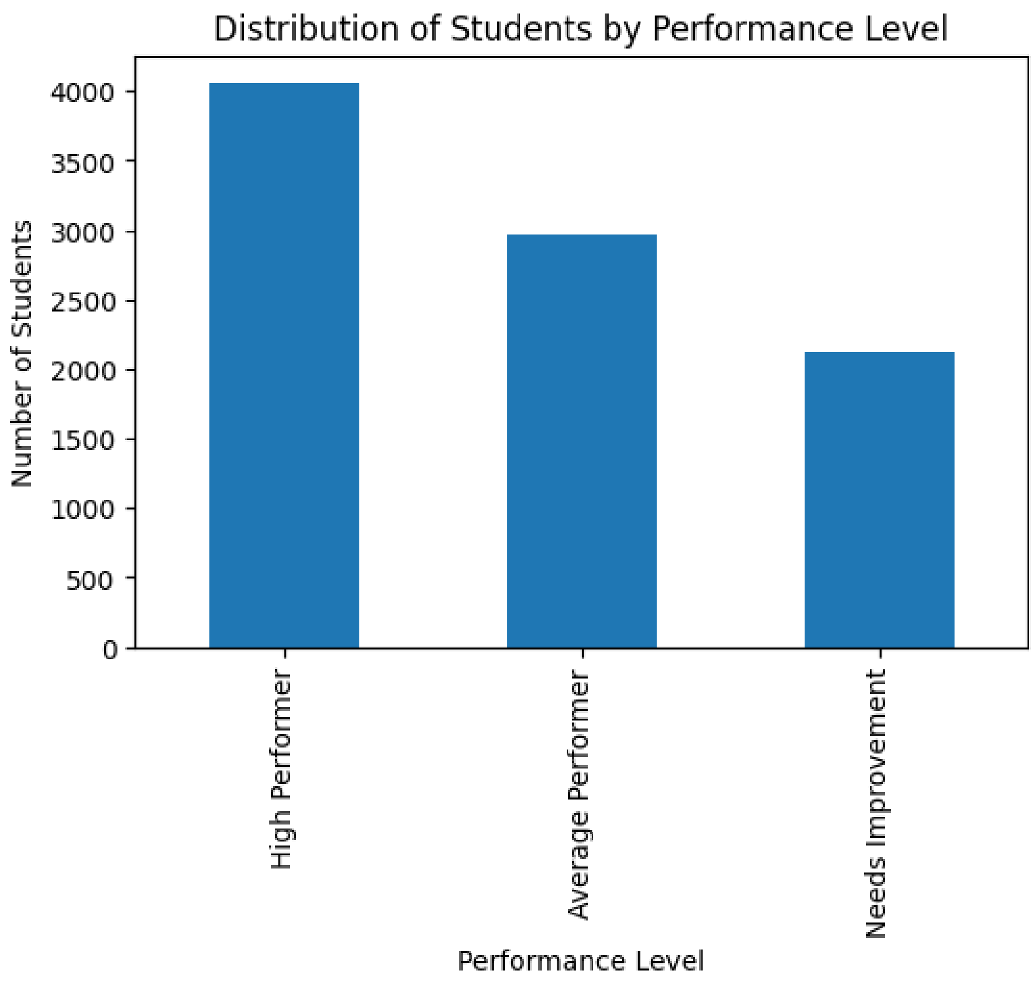
12.12 Plot Cluster Visualization :

```
plt.figure(figsize=(8, 5))

sns.scatterplot(
    x='PCA1',
    y='PCA2',
    hue='Cluster',
    data=df,
    palette='viridis'
)

plt.title("Cluster Visualization using PCA")

plt.show()
```



12.13 Performance Distribution Chart:

```
plt.figure(figsize=(8, 5))

sns.countplot(
    x='Performance',
    data=df,
    palette='Set2'
)

plt.title("Student Performance Distribution")

plt.show()
```



12.14 Evaluate Model using Silhouette Score :

```
import joblib

joblib.dump(kmeans, "kmeans_model.pkl")

df.to_csv("clustered_results.csv", index=False)
```

12.15 Save Model and Results :

```
import joblib

joblib.dump(kmeans, "kmeans_model.pkl")

df.to_csv("clustered_results.csv", index=False)
```

12.16 Student Performance Prediction Script :

```
import numpy as np
import pandas as pd
import joblib

# Load models
kmeans = joblib.load(
    r"D:\VNSGU_PDA2\Data-Analytics-using-Python-\
Minor-Project\models\kmeans_model.pkl"
)

scaler = joblib.load(
    r"D:\VNSGU_PDA2\Data-Analytics-using-Python-\
Minor-Project\models\scaler.pkl"
)

# Expected feature order
features = ["JAVA_Total", "NET_Total", "MAD_Total",
            "IOT_Total", "OSS_Total", "CC_Total",
            "BMP_Total"]

print("Enter Student Marks:")

new_student = {
    "JAVA_Total": float(input("JAVA Total: ")),
    "NET_Total": float(input("NET Total: ")),
    "MAD_Total": float(input("MAD Total: ")),
    "IOT_Total": float(input("IOT Total: ")),
    "OSS_Total": float(input("OSS Total: ")),
    "CC_Total": float(input("CC Total: ")),
    "BMP_Total": float(input("BMP Total: "))
}
```

```
# Convert input to DataFrame
new_df = pd.DataFrame([new_student])[features]

# Scale input
new_scaled = scaler.transform(new_df)

# Predict cluster
cluster = kmeans.predict(new_scaled)[0]

performance_map = {
    0: "Average Performer",
    1: "High Performer",
    2: "Needs Improvement"
}

print("\nPredicted Performance Level:",
      performance_map.get(cluster, "Unknown"))
```

```
PS D:\VNSGU_PDA2\Data-Analytics-using-Python-Minor-Project\src> python predict.py
Enter Student Marks:
JAVA Total: 89
NET Total: 98
MAD Total: 97
IOT Total: 95
OSS Total: 99
CC Total: 90
BMP Total: 86

Predicted Performance Level: High Performer
```

```
PS D:\VNSGU_PDA2\Data-Analytics-using-Python-Minor-Project\src> python predict.py
Enter Student Marks:
JAVA Total: 50
NET Total: 52
MAD Total: 53
IOT Total: 54
OSS Total: 53
CC Total: 56
BMP Total: 49

Predicted Performance Level: Average Performer
```

```
PS D:\VNSGU_PDA2\Data-Analytics-using-Python-Minor-Project\src> python predict.py
Enter Student Marks:
JAVA Total: 45
NET Total: 43
MAD Total: 46
IOT Total: 47
OSS Total: 43
CC Total: 42
BMP Total: 41

Predicted Performance Level: Needs Improvement
```

13.Conclusion

By applying different data analysis techniques and based on the statistical and visual results, we have reached the following conclusions:

1. The majority of students fall within the average academic performance range, indicating balanced overall academic results.
2. Most students scored between 55% and 75%, which represents moderate to good academic performance.
3. The pass rate is significantly higher than the fail rate, showing satisfactory academic outcomes in the dataset.
4. Network Technology (NT) has the highest number of failures, indicating that it is comparatively the most difficult subject for students.
5. Cloud Computing (CC) has the lowest number of failures, suggesting stronger student performance in this subject.
6. A strong positive correlation exists between subject marks, SGPA, and percentage, indicating that good performance in individual subjects contributes significantly to overall academic success.
7. Students who passed generally obtained higher total marks and percentages, while failing students are concentrated in the lower score ranges.
8. The K-Means clustering model successfully categorized students into three performance groups: High Performer, Average Performer, and Needs Improvement, helping to identify academic performance patterns among students

14.References

• Python Official Documentation
https://docs.python.org/3/

• Pandas Official Documentation
https://pandas.pydata.org/docs/

• NumPy Official Documentation
https://numpy.org/doc/

• Seaborn Documentation
https://seaborn.pydata.org/

• Matplotlib Documentation
https://matplotlib.org/stable/index.html

• Scikit-learn Documentation
https://scikit-learn.org/stable/

• VNSGU Student Result Portal (Dataset Source)
https://ums.vnsgu.net/Result/StudentResultDisplay.aspx?HtmlURL=7596,0000

• UCI Machine Learning Repository
https://archive.ics.uci.edu/

• GeeksforGeeks – Machine Learning Tutorials
https://www.geeksforgeeks.org/machine-learning/